RTL Design Sherpa

RTL AMBA APB5 Module Reference — Rev 0.90

July 2026

Table of Contents

1 APB5 (Advanced Peripheral Bus - AMBA 5) Modules

Location: `rtl/amba/apb5/` **Test Location:** `val/amba/` **Status:** Production Ready

---

1.1 Overview

The APB5 subsystem provides a complete implementation of the ARM AMBA 5 APB (Advanced Peripheral Bus) protocol, including masters, slaves, monitors, clock domain crossing, and testbench utilities.

APB5 extends APB4 with enhanced features for modern SoC designs while maintaining backward compatibility. The simple two-cycle handshake protocol is preserved, with additional signals for improved security, wake-up signaling, and error handling.

1.2 AMBA4 vs AMBA5 Comparison

The table below compares the APB4 and APB5 protocol definitions, and states what this RTL release actually implements. Optional APB5 features that are not implemented are simply absent from the module port lists – there are no tie-off ports to connect.

Feature	APB4	APB5	Implemented here
Basic Protocol	Two-phase handshake	Two-phase handshake	Yes
Protection	PPROT[2:0]	PPROT[2:0] + PNSE	PPROT only – no PNSE port
Wake-up	Not supported	PWAKEUP signal	Yes
User Signals	Not supported	PAUSER, PWUSER, PRUSER, PBUSER	Yes
Atomic Operations	Not supported	PEXCL, PEXOKAY	No – not in this release
Parity	Not supported	Optional signal parity	Yes (`ENABLE_PARITY`)
Error Response	PSLVERR	PSLVERR (enhanced semantics)	Yes

1.3 Module Categories

1.3.1 Core Protocol Components

Module	Description	Documentation	Status
apb5_master	Full-featured APB5 master with command/response interface	apb5_master.md	Documented
apb5_slave	Complete APB5 slave with buffered cmd/rsp interface	apb5_slave.md	Documented
apb5_slave_cdc	APB5 slave with clock domain crossing support	apb5_slave_cdc.md	Documented
apb5_monitor	Transaction monitoring with 64-bit monitor bus packet	apb5_monitor.md	Documented

1.3.2 Clock-Gated Variants

Module	Description	Documentation	Status
apb5_master_cg	APB5 master with integrated clock gating	apb5_master_cg.md	Documented
apb5_slave_cg	APB5 slave with integrated clock gating	apb5_slave_cg.md	Documented
apb5_slave_cdc_cg	APB5 slave CDC with integrated clock gating	apb5_slave_cdc_cg.md	Documented

1.3.3 Testbench Utilities

Module	Description	Documentation	Status
apb5_master_stub	Lightweight APB5 master for testbench integration	apb5_master_stub.md	Documented

Module	Description	Documentation	Status
apb5_slave_stub	Lightweight APB5 slave for testbench integration	apb5_slave_stub.md	Documented

1.4 Key Features

1.4.1 APB5 Protocol Support

- **PSTRB Support:** Byte-lane strobes for partial writes
- **PPROT Support:** Protection attributes for security-aware systems
- **PWAKEUP Support:** Low-power wake-up signaling
- **User Signals:** PAUSER, PWUSER, PRUSER, PBUSER for sideband data
- **Optional Parity:** Per-byte data parity plus address and control parity (`ENABLE_PARITY`)

Not implemented in this release: PNSE (Non-secure extension) and the PEXCL/PEXOKAY exclusive-access pair. No module in `rtl/amba/apb5/` declares these ports.

1.4.2 Clock Domain Crossing

- **Dual-Clock Operation:** APB (pclk) and backend (aclk) domains
- **Safe CDC:** Proper handshake-based clock domain crossing
- **Independent Frequencies:** Backend can run faster or slower than APB

1.4.3 Power Management

- **Clock Gating:** Per-module clock gating for power reduction
- **Wake-up Signaling:** PWAKEUP for low-power state exit
- **Idle Detection:** Automatic clock gate when bus is idle

1.4.4 Monitoring and Debug

- **Transaction Monitoring:** Real-time protocol monitoring
- **64-bit Monitor Bus:** Standardized packet format (`monbus_packet[63:0]`, no side-band timestamp)
- **Error Detection:** Protocol violations, timeout detection
- **Performance Tracking:** Transaction counting, latency measurement

1.5 Quick Start

1.5.1 Using APB5 Master

```
apb5_master #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .CMD_DEPTH (6),      // command FIFO entries: one of {2, 4, 6, 8}
    .RSP_DEPTH (6)       // response FIFO entries: one of {2, 4, 6, 8}
) u_apb5_master (
    .pclk           (clk),
    .presetn        (resetn),

    // Command interface
    .cmd_valid      (cmd_valid),
    .cmd_ready      (cmd_ready),
    .cmd_pwrite     (cmd_pwrite),
    .cmd_paddr      (cmd_paddr),
    .cmd_pwdata     (cmd_pwdata),
    .cmd_pstrb      (cmd_pstrb),
    .cmd_pprot      (cmd_pprot),
    .cmd_pauser     (cmd_pauser),
    .cmd_pwuser     (cmd_pwuser),

    // Response interface
    .rsp_valid      (rsp_valid),
    .rsp_ready      (rsp_ready),
    .rsp_prdata     (rsp_prdata),
    .rsp_pslverr    (rsp_pslverr),
    .rsp_pwakeup    (rsp_pwakeup),
    .rsp_pruser     (rsp_pruser),
    .rsp_pbuser     (rsp_pbuser),

    // APB5 master interface
    .m_apb_PSEL     (psel),
    .m_apb_PENABLE  (penable),
    .m_apb_PREADY   (pready),
    .m_apb_PADDR    (paddr),
    .m_apb_PWRITE   (pwrite),
    .m_apb_PWDATA   (pwdata),
    .m_apb_PSTRB    (pstrb),
    .m_apb_PPROT    (pprot),
    .m_apb_PAUSER   (pauser),
    .m_apb_PWUSER   (pwuser),
    .m_apb_PRDATA   (prdata),
    .m_apb_PSLVERR  (pslverr),
    .m_apb_PWAKEUP  (pwakeup),      // input: driven by the slave
    .m_apb_PRUSER   (pruser),
```

```
    .m_apb_PBUSER   (pbuser),

    // Parity (tie off / leave open when ENABLE_PARITY=0)
    .m_apb_PWDATAPARITY  (),
    .m_apb_PADDRPARITY   (),
    .m_apb_PCTRLPARITY   (),
    .m_apb_PRDATAPARITY  ('0),
    .m_apb_PREADYPARITY  (1'b0),
    .m_apb_PSLVERRPARITY (1'b0),
    .parity_error_rdata  (),
    .parity_error_ctrl   (),

    // Status
    .wakeup_pending (wakeup_pending)
);
```

1.5.2 Using APB5 Slave

```
apb5_slave #(
    .ADDR_WIDTH(32),
    .DATA_WIDTH(32),
    .DEPTH(2)
) u_apb5_slave (
    .pclk           (clk),
    .presetn        (resetn),

    // APB5 slave interface
    .s_apb_PSEL     (psel),
    .s_apb_PENABLE  (penable),
    .s_apb_PREADY   (pready),
    .s_apb_PADDR    (paddr),
    .s_apb_PWRITE   (pwrite),
    .s_apb_PWDATA   (pwdata),
    .s_apb_PSTRB    (pstrb),
    .s_apb_PPROT    (pprot),
    .s_apb_PAUSER   (pauser),
    .s_apb_PWUSER   (pwuser),
    .s_apb_PRDATA   (prdata),
    .s_apb_PSLVERR  (pslverr),
    .s_apb_PWAKEUP  (pwakeup),      // output: driven to the master
    .s_apb_PRUSER   (pruser),
    .s_apb_PBUSER   (pbuser),

    // Command interface
    .cmd_valid      (cmd_valid),
    .cmd_ready      (cmd_ready),
    .cmd_pwrite     (cmd_pwrite),
    .cmd_paddr      (cmd_paddr),
```

```
    .cmd_pwdata     (cmd_pwdata),
    .cmd_pstrb      (cmd_pstrb),
    .cmd_pprot      (cmd_pprot),
    .cmd_pauser     (cmd_pauser),
    .cmd_pwuser     (cmd_pwuser),

    // Response interface
    .rsp_valid      (rsp_valid),
    .rsp_ready      (rsp_ready),
    .rsp_prdata     (rsp_prdata),
    .rsp_pslverr    (rsp_pslverr),
    .rsp_pruser     (rsp_pruser),
    .rsp_pbuser     (rsp_pbuser),

    // Wake-up request from the backend (drives s_apb_PWAKEUP)
    .wakeup_request (wakeup_request),

    // Parity (tie off / leave open when ENABLE_PARITY=0)
    .s_apb_PWDATAPARITY  ('0),
    .s_apb_PADDRPARITY   (1'b0),
    .s_apb_PCTRLPARITY   (1'b0),
    .s_apb_PRDATAPARITY  (),
    .s_apb_PREADYPARITY  (),
    .s_apb_PSLVERRPARITY (),
    .parity_error_wdata  (),
    .parity_error_ctrl   ()
);
```

1.6 Testing

All APB5 modules are verified using CocoTB-based testbenches located in `val/amba/`:

```
# Run all APB5 tests
pytest val/amba/test_apb5*.py -v

# Run specific module tests
pytest val/amba/test_apb5_master.py -v
pytest val/amba/test_apb5_slave.py -v
pytest val/amba/test_apb5_slave_cdc.py -v
pytest val/amba/test_apb5_monitor.py -v
```

1.7 Protocol Details

1.7.1 APB5 Transfer Phases

APB5 uses the same two-phase protocol as APB4:

1. **SETUP Phase:** PSEL asserted, PENABLE low
   - Master presents address, control signals, and write data (if write)
   - Slave prepares for transfer
2. **ACCESS Phase:** PSEL and PENABLE both asserted
   - Slave completes transfer and asserts PREADY when ready
   - For reads, slave presents PRDATA
   - Slave may assert PSLVERR to signal error

1.7.2 APB5 Signal Descriptions

In this implementation the clock and reset ports are named `pclk` and `presetn`, and the bus signals are prefixed `m_apb_` on masters and `s_apb_` on slaves (for example `m_apb_PADDR`, `s_apb_PREADY`).

Signal	Direction	Description
pclk	Input	APB clock
presetn	Input	Active-low reset
PSEL	Master to Slave	Slave select
PENABLE	Master to Slave	Enable (ACCESS phase indicator)
PREADY	Slave to Master	Transfer complete
PADDR	Master to Slave	Address bus
PWRITE	Master to Slave	Write enable (1=write, 0=read)
PWDATA	Master to Slave	Write data
PSTRB	Master to Slave	Byte lane strobes
PPROT	Master to Slave	Protection attributes
PWAKEUP	Slave to Master	Wake-up signal (APB5) – see note below
PAUSER	Master to Slave	Address phase user

Signal	Direction	Description
		signal (APB5)
PWUSER	Master to Slave	Write data user signal (APB5)
PRDATA	Slave to Master	Read data
PSLVERR	Slave to Master	Error response
PRUSER	Slave to Master	Read data user signal (APB5)
PBUSER	Slave to Master	Write response user signal (APB5)

PWAKEUP direction note: this suite implements PWAKEUP as a slave-to-master signal used by a peripheral to request that the master (and its power domain) stay awake. `apb5_slave` drives `s_apb_PWAKEUP` from its `wakeup_request` input and `apb5_master` consumes `m_apb_PWAKEUP` as an input, capturing it in the response packet (`rsp_pwakeup`) and in the `wakeup_pending` status output.

PNSE, PEXCL and PEXOKAY are not implemented and do not appear on any module port list.

1.7.3 Optional Parity Signals

Present on all masters and slaves; meaningful only when `ENABLE_PARITY=1`.

Signal	Width	Direction	Description
PWDATAPARITY	STRB_WIDTH	Master to Slave	One parity bit per write-data byte lane
PADDRPARITY	1	Master to Slave	Single parity bit over the whole address
PCTRLPARITY	1	Master to Slave	Single parity bit over {PWRITE, PSTRB, PPROT}
PRDATAPARITY	STRB_WIDTH	Slave to Master	One parity bit per read-data byte lane
PREADYPARITY	1	Slave to Master	Parity bit for PREADY
PSLVERRPARITY	1	Slave to Master	Parity bit for PSLVERR

1.8 Design Notes

1.8.1 Command/Response Architecture

All APB5 modules use a command/response interface pattern for backend integration, identical to the APB4 architecture:

Command Interface: - `cmd_valid`, `cmd_ready` - Handshake signals - `cmd_pwrite` - Write/read direction - `cmd_paddr` - Transaction address - `cmd_pwdata` - Write data - `cmd_pstrb` - Byte strobes - `cmd_pprot` - Protection attributes - `cmd_pauser`, `cmd_pwuser` - APB5 user attributes

Response Interface: - `rsp_valid`, `rsp_ready` - Handshake signals - `rsp_prdata` - Read data - `rsp_pslverr` - Error flag - `rsp_pruser`, `rsp_pbuser` - APB5 user attributes - `rsp_pwakeup` - Captured PWAKEUP state (master only)

1.8.2 Migration from APB4

APB5 modules are backward compatible with APB4 systems: - Connect APB5 signals to APB4 equivalents - Tie unused APB5 inputs (PWAKEUP, user signals, parity) to default values and leave the corresponding outputs unconnected - On a master, tie `m_apb_PWAKEUP` to 0 for always-awake operation - On a slave, tie `wakeup_request` to 0 so `s_apb_PWAKEUP` stays low - Leave `ENABLE_PARITY=0` (the default), which forces all generated parity outputs and both `parity_error_*` flags to 0

1.9 Related Documentation

- **APB4 Modules** - APB4 protocol components
- **AXI5 Modules** - AXI5 protocol components
- **AXIS5 Modules** - AXI5-Stream components
- **GAXI Modules** - Generic AXI utilities

1.10 References

1.10.1 Specifications

- ARM AMBA 5 APB Protocol Specification
- ARM AMBA APB Protocol Specification v2.0 (APB4)

1.10.2 Source Code

- RTL: `rtl/amba/apb5/`
- Tests: `val/amba/test_apb5*.py`

- Framework: `bin/TBClasses/components/apb/`

Last Updated: 2026-07-19

1.11 Navigation

- **Back to RTLAmba Index**
- **Back to Main Documentation Index**

2 APB5 Master

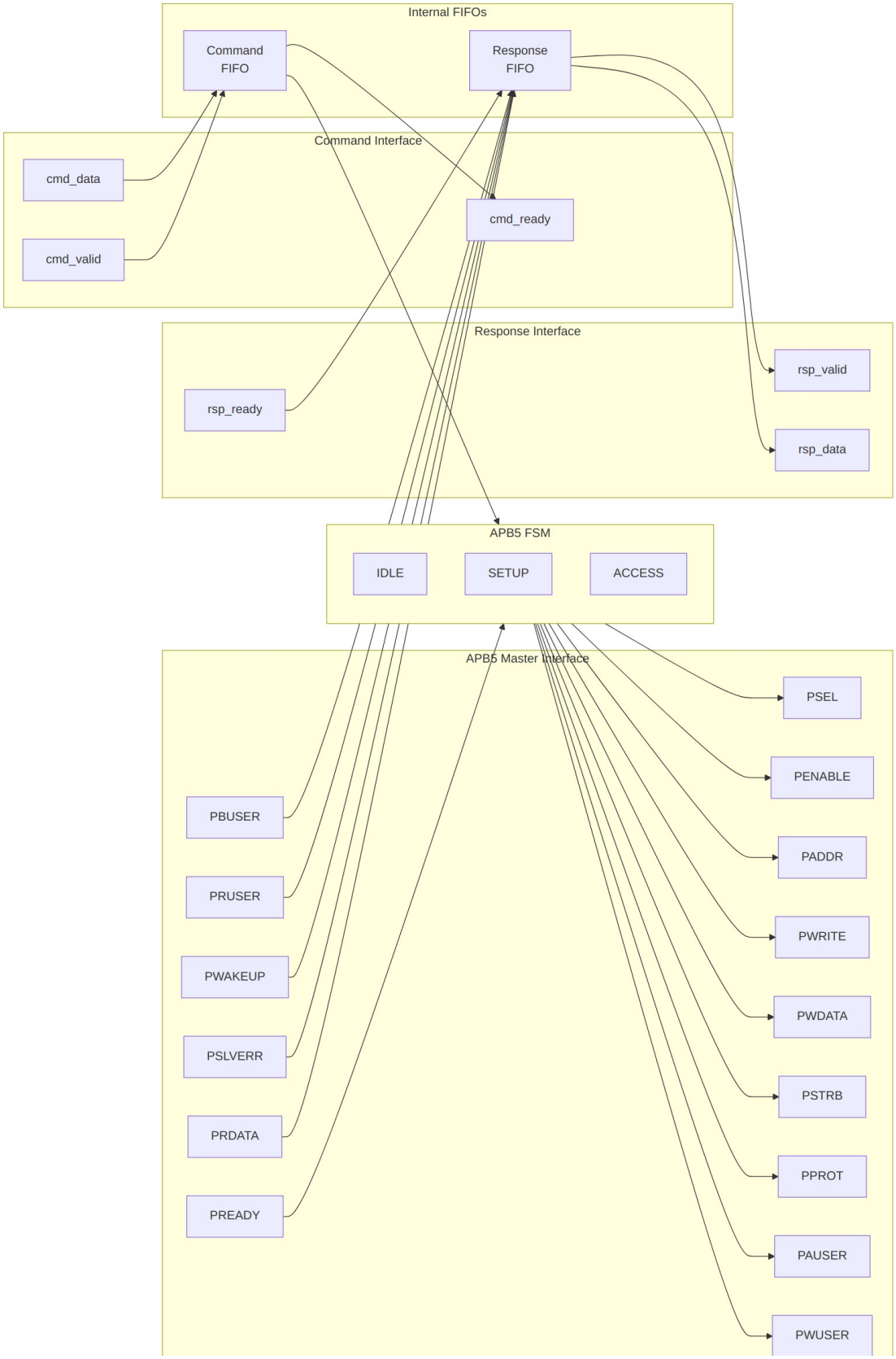
Module: `apb5_master.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

2.1 Overview

The APB5 Master module implements a complete AMBA APB5 master interface with all APB5 extensions including user-defined signals, wake-up support, and optional parity for data integrity. It provides command/response buffering for improved system performance.

2.1.1 Key Features

- Full AMBA APB5 protocol compliance
- PAUSER: User-defined request attributes
- PWUSER: User-defined write data attributes
- PRUSER/PBUSER: User-defined response attributes from slave
- PWAKEUP: Wake-up signal handling from slave
- Optional parity support for data integrity
- Command and response FIFO buffering
- Configurable FIFO depths


Internal FIFOs
Command FIFO
Response FIFO
Command Interface
cmd_data
cmd_valid
cmd_ready
Response Interface
rsp_ready
rsp_valid
rsp_data
APB5 FSM
IDLE
SETUP
ACCESS
APB5 Master Interface
PBUSER
PRUSER
PWAKEUP
PSLVERR
PRDATA
PREADY
PSEL
PENABLE
PADDR
PWRITE
PWDATA
PSTRB
PPROT
PAUSER
PWUSER

2.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address/request user signal width
WUSER_WIDTH	int	4	Write data user signal width
RUSER_WIDTH	int	4	Read data user signal width
BUSER_WIDTH	int	4	Response user signal width
CMD_DEPTH	int	6	Command skid-buffer depth in entries; must be one of {2, 4, 6, 8}
RSP_DEPTH	int	6	Response skid-buffer depth in entries; must be one of {2, 4, 6, 8}
ENABLE_PARITY	bit	0	Enable parity generation and checking
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)

`CMD_DEPTH` and `RSP_DEPTH` are passed straight through to `gaxi_skid_buffer`, whose `DEPTH` is a literal entry count – not a log2 exponent. The default of 6 means six buffered entries.

Two further computed parameters are exposed for reference and should not be overridden:

Parameter	Formula	Default value
CPW	ADDR_WIDTH + DATA_WIDTH + STRB_WIDTH +	80

Parameter	Formula	Default value
	PROT_WIDTH + AUSER_WIDTH + WUSER_WIDTH + 1	
RPW	DATA_WIDTH + RUSER_WIDTH + BUSER_WIDTH + 2	42

2.3 Ports

2.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB active-low reset

2.3.2 APB5 Master Interface

Port	Width	Direction	Description
m_apb_PSEL	1	Output	APB select signal
m_apb_PENABLE	1	Output	APB enable signal
m_apb_PADDR	ADDR_WIDTH	Output	APB address
m_apb_PWRITE	1	Output	Write/read indicator (1=write)
m_apb_PWDATA	DATA_WIDTH	Output	Write data
m_apb_PSTRB	STRB_WIDTH	Output	Write byte strobes
m_apb_PPROT	PROT_WIDTH	Output	Protection attributes
m_apb_PA	AUSER_WIDT	Output	User-defined request

Port	Width	Direction	Description
USER	H		attributes
m_apb_PWUSER	WUSER_WIDTH	Output	User-defined write data attributes
m_apb_PRDATA	DATA_WIDTH	Input	Read data from slave
m_apb_PSLVERR	1	Input	Slave error response
m_apb_PREADY	1	Input	Slave ready
m_apb_PWAKEUP	1	Input	Wake-up signal from slave
m_apb_PRUSER	RUSER_WIDTH	Input	User-defined read data attributes
m_apb_PBUSER	BUSER_WIDTH	Input	User-defined response attributes

2.3.3 Parity Signals (Optional)

Port	Width	Direction	Description
m_apb_PWDATAPARITY	STRB_WIDTH	Output	Write data parity (per byte)
m_apb_PADDRPARITY	1	Output	Address parity
m_apb_PCTRLPARITY	1	Output	Control signals parity
m_apb_PRDATAPARITY	STRB_WIDTH	Input	Read data parity from slave
m_apb_PREADYPARITY	1	Input	PREADY parity from slave
m_apb_PSL	1	Input	PSLVERR parity from

Port	Width	Direction	Description
VERRPARITY			slave

2.3.4 Command Interface

Port	Width	Direction	Description
cmd_valid	1	Input	Command valid
cmd_ready	1	Output	Command ready (FIFO not full)
cmd_pwrite	1	Input	Command write/read
cmd_paddr	ADDR_WIDTH	Input	Command address
cmd_pwdata	DATA_WIDTH	Input	Command write data
cmd_pstrb	STRB_WIDTH	Input	Command write strobes
cmd_pprot	PROT_WIDTH	Input	Command protection attributes
cmd_pauser	AUSER_WIDTH	Input	Command user attributes
cmd_pwuser	WUSER_WIDTH	Input	Command write user attributes

2.3.5 Response Interface

Port	Width	Direction	Description
rsp_valid	1	Output	Response valid
rsp_ready	1	Input	Response ready
rsp_prdata	DATA_WIDTH	Output	Response read data

Port	Width	Direction	Description
rsp_pslverr	1	Output	Response error status
rsp_pwakeup	1	Output	Response wake-up indicator
rsp_pruser	RUSER_WIDTH	Output	Response read user attributes
rsp_pbuser	BUSER_WIDTH	Output	Response user attributes

2.3.6 Status Outputs

Port	Width	Direction	Description
parity_error_rdata	1	Output	Read-data parity mismatch (tied to 0 when ENABLE_PARITY=0)
parity_error_ctrl	1	Output	PREADY/PSLVERR parity mismatch (tied to 0 when ENABLE_PARITY=0)
wakeup_pending	1	Output	Sticky flag: PWAKEUP was seen and no transaction has started since

2.4 Functionality

```
stateDiagram-v2
    [*] --> IDLE

    IDLE --> SETUP : cmd_fifo_valid
    SETUP --> ACCESS : always
    ACCESS --> IDLE : PREADY & cmd_fifo_empty
    ACCESS --> SETUP : PREADY & !cmd_fifo_empty

    state IDLE {
        note right of IDLE : PSEL=0, PENABLE=0
    }
    state SETUP {
        note right of SETUP : PSEL=1, PENABLE=0
```

```
    }
    state ACCESS {
        note right of ACCESS : PSEL=1, PENABLE=1
    }
```

2.4.1 APB5 Extensions

User Signals: - **PAUSER**: Carries user-defined attributes with the address/control phase - **PWUSER**: Carries user-defined attributes with write data - **PRUSER**: Returns user-defined attributes with read data - **PBUSER**: Returns user-defined attributes with the response

Wake-up Support: - **PWAKEUP**: Slave can assert to indicate wake-up events - Captured in response packet (`rsp_pwakeup`) for software handling - Also latched into `wakeup_pending`, which sets on any PWAKEUP pulse and clears once the FSM leaves IDLE to start a transaction

Parity Protection: - Optional parity on data, address, and control signals - Enables detection of single-bit transmission errors

See Parity Implementation for the exact coverage of each parity bit.

2.5 Timing Diagrams

2.5.1 Basic Write Transaction

Timing diagram pending. The signals and sequence this scenario exercises:

- PCLK
- PSEL
- PENABLE
- PADDR
- PWRITE (high)
- PWDATA
- PSTRB
- PAUSER
- PWUSER
- PREADY
- PSLVERR

2.5.2 Basic Read Transaction

Timing diagram pending. The signals and sequence this scenario exercises:

- PCLK
- PSEL
- PENABLE
- PADDR
- PWRITE (low)
- PAUSER
- PREADY
- PRDATA
- PRUSER
- PSLVERR

2.5.3 Wake-up Signal Handling

Timing diagram pending. The signals and sequence this scenario exercises:

- PCLK
- Transaction signals
- PWAKEUP assertion
- Response capture

2.6 Usage Example

```
apb5_master #(
    .ADDR_WIDTH     (32),
    .DATA_WIDTH     (32),
    .AUSER_WIDTH    (4),
    .WUSER_WIDTH    (4),
    .RUSER_WIDTH    (4),
    .BUSER_WIDTH    (4),
    .CMD_DEPTH      (4),
    .RSP_DEPTH      (4),
    .ENABLE_PARITY  (0)
) u_apb5_master (
    .pclk           (apb_clk),
    .presetn        (apb_rst_n),
```

```
// APB5 master interface
.m_apb_PSEL     (m_apb_psel),
.m_apb_PENABLE  (m_apb_penable),
.m_apb_PADDR    (m_apb_paddr),
.m_apb_PWRITE   (m_apb_pwrite),
.m_apb_PWDATA   (m_apb_pwdata),
.m_apb_PSTRB    (m_apb_pstrb),
.m_apb_PPROT    (m_apb_pprot),
.m_apb_PAUSER   (m_apb_pauser),
.m_apb_PWUSER   (m_apb_pwuser),
.m_apb_PRDATA   (m_apb_prdata),
.m_apb_PSLVERR  (m_apb_pslverr),
.m_apb_PREADY   (m_apb_pready),
.m_apb_PWAKEUP  (m_apb_pwakeup),
.m_apb_PRUSER   (m_apb_pruser),
.m_apb_PBUSER   (m_apb_pbuser),

// Command interface
.cmd_valid      (cmd_valid),
.cmd_ready      (cmd_ready),
.cmd_pwrite     (cmd_write),
.cmd_paddr      (cmd_addr),
.cmd_pwdata     (cmd_wdata),
.cmd_pstrb      (cmd_strb),
.cmd_pprot      (cmd_prot),
.cmd_pauser     (cmd_auser),
.cmd_pwuser     (cmd_wuser),

// Response interface
.rsp_valid      (rsp_valid),
.rsp_ready      (rsp_ready),
.rsp_prdata     (rsp_rdata),
.rsp_pslverr    (rsp_error),
.rsp_pwakeup    (rsp_wakeup),
.rsp_pruser     (rsp_ruser),
.rsp_pbuser     (rsp_buser),

// Parity interface (unused here because ENABLE_PARITY=0)
.m_apb_PWDATAPARITY  (),
.m_apb_PADDRPARITY   (),
.m_apb_PCTRLPARITY   (),
.m_apb_PRDATAPARITY  ('0),
.m_apb_PREADYPARITY  (1'b0),
.m_apb_PSLVERRPARITY (1'b0),
.parity_error_rdata  (),
.parity_error_ctrl   (),
```

```
    // Status
    .wakeup_pending (apb_wakeup_pending)
);
```

2.7 Design Notes

2.7.1 APB5 vs APB4 Differences

Feature	APB4	APB5	Implemented here
User signals	None	PAUSER, PWUSER, PRUSER, PBUSER	Yes
Wake-up	None	PWAKEUP	Yes (slave to master)
Parity	None	Optional signal parity	Yes (`ENABLE_PARITY`)
Non-secure extension	None	PNSE	No – no port
Exclusive access	None	PEXCL, PEXOKAY	No – no ports

2.7.2 FIFO Sizing

- Command depth should match the expected command burst length
- Response depth should match to prevent backpressure
- `CMD_DEPTH` / `RSP_DEPTH` are entry counts, restricted to {2, 4, 6, 8} by the underlying `gaxi_skid_buffer`

2.7.3 Parity Implementation

When `ENABLE_PARITY=1` the master generates parity on its outgoing signals and checks parity on the signals returned by the slave. Coverage differs per signal group, which determines what a single parity bit can detect:

Parity signal	Covers	Granularity
m_apb_PWDATAPARITY[i]	PWDATA byte	One bit per byte

Parity signal	Covers	Granularity
	lane `i`	(STRB_WIDTH bits total)
m_apb_PADDRPARITY	Whole PADDR	One bit for the entire address
m_apb_PCTRLPARITY	{PWRITE, PSTRB, PPROT} concatenated	One bit for the whole control group
m_apb_PRDATAPARITY[i]	PRDATA byte lane `i` (checked)	One bit per byte
m_apb_PREADYPARITY	PREADY (checked)	One bit
m_apb_PSLVERRPARITY	PSLVERR (checked)	One bit

Each parity bit is the XOR reduction of the covered signals, so it is 1 when the covered field contains an odd number of ones (an even-parity encoding: field plus parity bit always has an even number of ones). Per-byte data parity detects one bit error in each byte independently; the single address and control bits detect only an odd number of errors across the whole group.

Checking is purely combinational and qualified by the bus state: `parity_error_rdata` and `parity_error_ctrl` are only asserted while `m_apb_PREADY && m_apb_PSEL && m_apb_PENABLE`, and read 0 otherwise. Downstream error handling is system-specific – the master does not itself abort or retry a transfer on a parity mismatch. Parity adds combinational logic but no pipeline stages, so it costs no additional latency.

2.8 Related Documentation

- **APB5 Slave** - APB5 slave interface
- **APB5 Master CG** - Clock-gated variant
- **APB5 Monitor** - Protocol monitor
- **APB4 Master** - APB4 version for comparison

2.9 Navigation

- **← Back to APB5 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

3 APB5 Master (Clock-Gated)

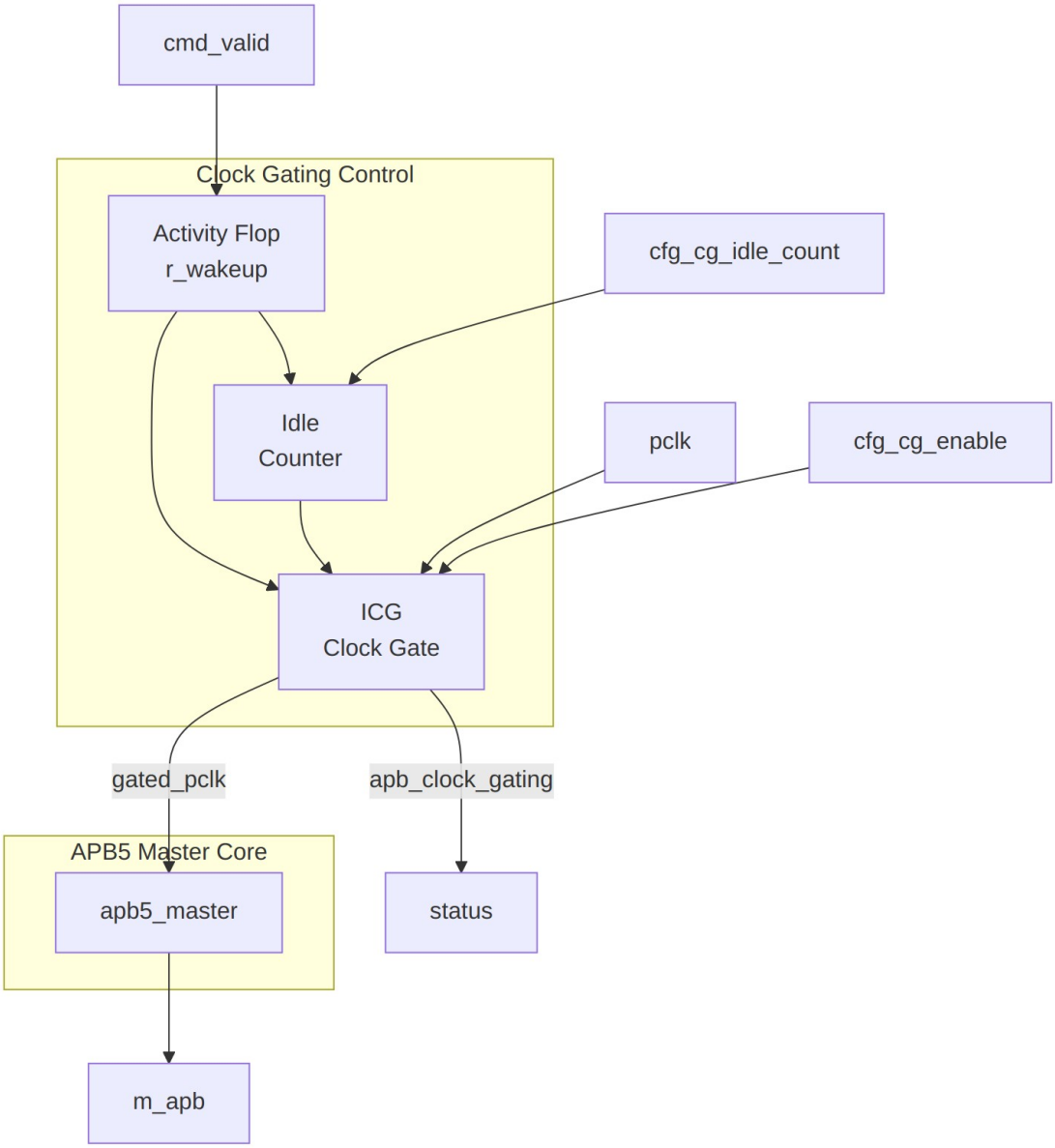
Module: `apb5_master_cg.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

3.1 Overview

Clock-gated variant of the APB5 Master module. Wraps the base `apb5_master` with clock gating control logic to reduce dynamic power consumption during idle periods.

3.1.1 Key Features

- All APB5 Master features (see apb5_master)
- Automatic clock gating during idle periods
- Configurable idle threshold before gating
- Registered wake-up on new activity, through a glitch-free ICG cell
- Power consumption reduction for low-duty-cycle applications



Module Architecture

3.2 Additional Parameters

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter (max idle = 2^N-1 cycles)

All other parameters inherited from apb5_master.

3.3 Additional Ports

3.3.1 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Enable clock gating (0=disabled, clock always runs)
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating

3.3.2 Clock Gating Status

Port	Width	Direction	Description
apb_clock_gating	1	Output	High while the internal clock is gated off

There is no cumulative gated-cycle counter port on this module. If a gated-cycle total is needed, count `apb_clock_gating` in the integrating logic on the ungated `pclk`.

All ports of apb5_master – including the parity signals, `parity_error_rdata`, `parity_error_ctrl` and `wakeup_pending` – are present unchanged and pass straight through to the wrapped core.

3.4 Clock Gating Behavior

```
stateDiagram-v2
    [*] --> RUNNING

    RUNNING --> COUNTING : No activity
    COUNTING --> RUNNING : Activity detected
    COUNTING --> GATED : count >= threshold
    GATED --> RUNNING : cmd_valid or activity

    state RUNNING {
        note right of RUNNING : Clock running
    }
    state COUNTING {
        note right of COUNTING : Counting idle cycles
    }
    state GATED {
```

```
        note right of GATED : Clock stopped
    }
```

3.4.1 Activity Detection and Wake-up Latency

The wrapper keeps the clock running whenever any of the following is high:

```
cmd_valid || rsp_valid || m_apb_PSEL || m_apb_PENABLE || m_apb_PWAKEUP
```

That term is registered into `r_wakeup` on the ungated `pclk`, and `amba_clock_gate_ctrl` registers it once more before it reaches the gating condition. Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream) before reaching the ICG enable, which is combinational. APB5 is a **two-stage** family, so the first gated-clock rising edge available to the wrapped `apb5_master` arrives **3 ungated `pclk` cycles** after activity asserts. Nothing is lost during those cycles – the core is simply held static, and the APB transfer stretches by that many wait states.

The actual gating element is an ICG (integrated clock gating) cell instantiated by `clock_gate_ctrl`, not a bare AND gate, so enable changes are glitch-free. Note that ICG cells are an ASIC-library primitive; on FPGA targets a clock-enable approach should be used instead.

Gating engages `cfg_cg_idle_count + 1` ungated `pclk` cycles after the internal wakeup deasserts, which is `cfg_cg_idle_count + 3` cycles after the last bus activity, because APB5 adds two register stages ahead of the ICG enable. With the default `CG_IDLE_COUNT_WIDTH=4` the maximum programmable idle threshold is 15 cycles.

3.4.2 Timing

Timing diagram pending. The signals and sequence this scenario exercises:

- pclk
- gated_pclk
- cmd_valid
- cfg_cg_idle_count
- idle_counter
- apb_clock_gating
- Transaction before/after gating

3.5 Usage Example

```
apb5_master_cg #(
    .ADDR_WIDTH         (32),
    .DATA_WIDTH         (32),
    .AUSER_WIDTH        (4),
```

```
    .WUSER_WIDTH        (4),
    .CG_IDLE_COUNT_WIDTH(4)
) u_apb5_master_cg (
    .pclk               (apb_clk),
    .presetn            (apb_rst_n),

    // Clock gating configuration
    .cfg_cg_enable      (1'b1),
    .cfg_cg_idle_count  (4'd8),    // Gate after 8 idle cycles

    // Clock gating status
    .apb_clock_gating   (master_clk_gated),

    // APB5 and command/response interfaces
    // ... (same as apb5_master)
);
```

3.6 Power Savings

The figures below are first-order expectations derived from the fraction of cycles the clock is gated off; they are analytical estimates, not measured silicon or post-layout power numbers. Actual savings depend on the technology library, the ICG cell, and the clock-tree share of total dynamic power.

Traffic Pattern	Duty Cycle	Estimated Dynamic Savings
Burst	30%	35-40%
Mixed	50%	20-25%
Continuous	90%+	<5%

3.7 Related Documentation

- **APB5 Master** - Base module documentation
- **APB5 Slave CG** - Clock-gated slave
- **Clock Gating Guide** - General clock gating concepts

3.8 Navigation

- **← Back to APB5 Index**

- ← Back to RTLAmba Index
- ← Back to Main Documentation Index

4 APB5 Master Stub

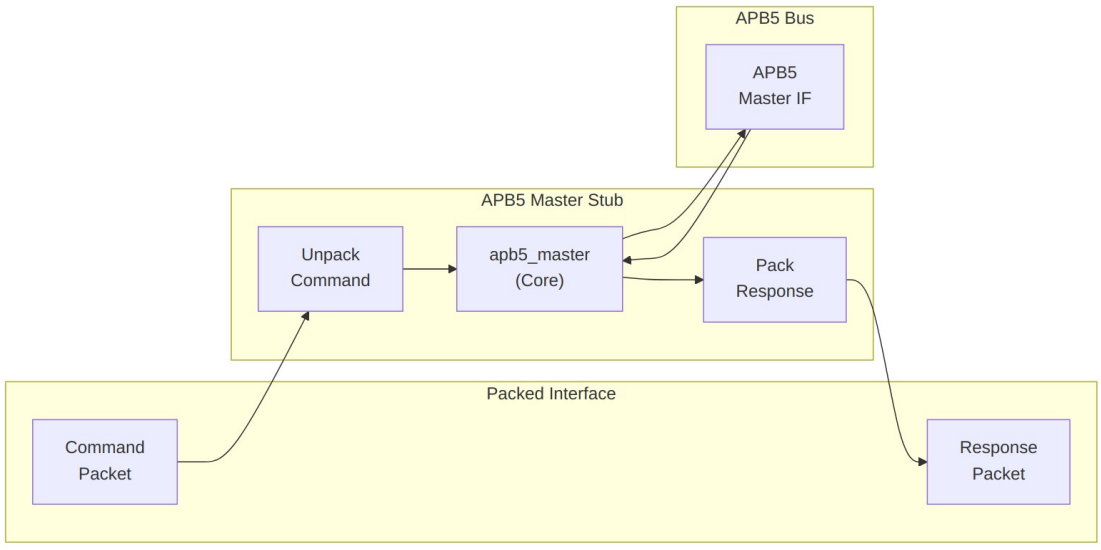
Module: `apb5_master_stub.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

4.1 Overview

The APB5 Master Stub provides a simplified packed-data interface for driving APB5 transactions. It wraps the full `apb5_master` module with packet-based command/response interfaces, making it ideal for testbenches and integration scenarios where a simpler interface is preferred.

4.1.1 Key Features

- Packed command/response packet interface
- Wraps fully-tested `apb5_master` internally
- All APB5 extensions (PAUSER, PWUSER, PRUSER, PBUSER)
- PWAKEUP signal handling from slave
- Optional parity support
- First/last markers for transaction tracking



Module Architecture

4.2 Parameters

Parameter	Type	Default	Description
CMD_DEPTH	int	6	Command skid-buffer depth in entries; one of {2, 4, 6, 8}
RSP_DEPTH	int	6	Response skid-buffer depth in entries; one of {2, 4, 6, 8}
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address user signal width
WUSER_WIDTH	int	4	Write user signal width
RUSER_WIDTH	int	4	Read user signal width
BUSER_WIDTH	int	4	Response user signal width
ENABLE_PARITY	bit	0	Enable parity signals
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width
CMD_PACKET_WIDTH	int	(calculated)	Total command packet width
RESP_PACKET_WIDTH	int	(calculated)	Total response packet width

`CMD_PACKET_WIDTH` and `RESP_PACKET_WIDTH` are computed from the width parameters and must not be overridden:

```
CMD_PACKET_WIDTH  = ADDR_WIDTH + DATA_WIDTH + STRB_WIDTH + PROT_WIDTH
                  + AUSER_WIDTH + WUSER_WIDTH + 1 + 1 + 1
                    // the three single bits are pwrite, first, last

RESP_PACKET_WIDTH = DATA_WIDTH + RUSER_WIDTH + BUSER_WIDTH + 1 + 1 + 1
+ 1
                    // the four single bits are pslverr, pwakeup,
first, last
```

With all defaults (32-bit address and data, 4-bit strobe, 3-bit protection, 4-bit user fields) this gives `CMD_PACKET_WIDTH = 82` and `RESP_PACKET_WIDTH = 44`.

4.3 Ports

4.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB reset (active low)

4.3.2 APB5 Master Interface

Port	Width	Direction	Description
m_apb_PSEL	1	Output	APB select signal
m_apb_PENABLE	1	Output	APB enable signal
m_apb_PADDR	ADDR_WIDTH	Output	APB address
m_apb_PWRITE	1	Output	Write/read indicator
m_apb_PWDATA	DATA_WIDTH	Output	Write data
m_apb_PSTRB	STRB_WIDTH	Output	Write byte strobes
m_apb_PPROT	PROT_WIDTH	Output	Protection attributes
m_apb_PAUSER	AUSER_WIDTH	Output	User request attributes
m_apb_PWUSER	WUSER_WIDTH	Output	User write attributes
m_apb_PRDATA	DATA_WIDTH	Input	Read data from slave
m_apb_PSLVERR	1	Input	Slave error response

Port	Width	Direction	Description
m_apb_PREADY	1	Input	Slave ready
m_apb_PWAKEUP	1	Input	Wake-up from slave
m_apb_PRUSER	RUSER_WIDTH	Input	User read attributes
m_apb_PBUSER	BUSER_WIDTH	Input	User response attributes

4.3.3 Parity Signals (Optional)

Present unconditionally; meaningful only when `ENABLE_PARITY=1`. Semantics are identical to apb5_master.

Port	Width	Direction	Description
m_apb_PWDATAPARITY	STRB_WIDTH	Output	Write data parity (per byte)
m_apb_PADDRPARITY	1	Output	Address parity
m_apb_PCTRLPARITY	1	Output	Control signals parity
m_apb_PRDATAPARITY	STRB_WIDTH	Input	Read data parity from slave
m_apb_PREADYPARITY	1	Input	PREADY parity from slave
m_apb_PSLVERRPARITY	1	Input	PSLVERR parity from slave

4.3.4 Command Packet Interface

Port	Width	Direction	Description
cmd_valid	1	Input	Command packet valid
cmd_ready	1	Output	Ready to accept command
cmd_data	CMD_PACKET_WIDTH	Input	Packed command data

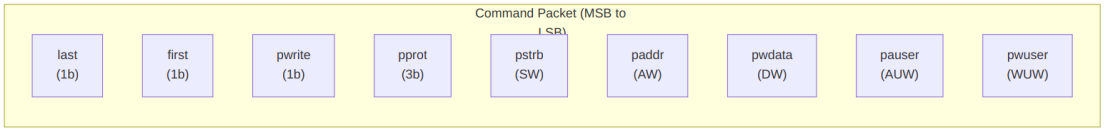
4.3.5 Response Packet Interface

Port	Width	Direction	Description
rsp_valid	1	Output	Response packet valid
rsp_ready	1	Input	Ready to accept response
rsp_data	RESP_PACKET_WIDTH	Output	Packed response data

4.3.6 Status Outputs

Port	Width	Direction	Description
parity_error_rdata	1	Output	Read data parity error
parity_error_ctrl	1	Output	Control signal parity error
wakeup_pending	1	Output	Wake-up signal active

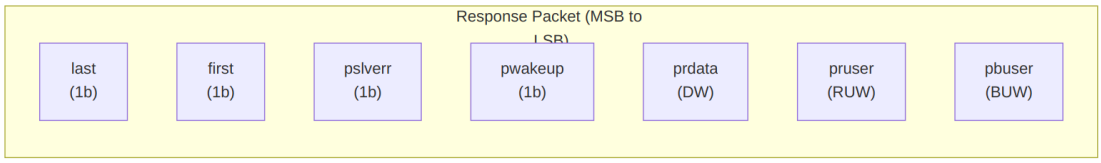
4.4 Packet Formats



Command Packet Structure

Bit Positions:

```
cmd_data = {last, first, pwrite, pprot, pstrb, paddr, pwdata, pauser,
pwuser}
```

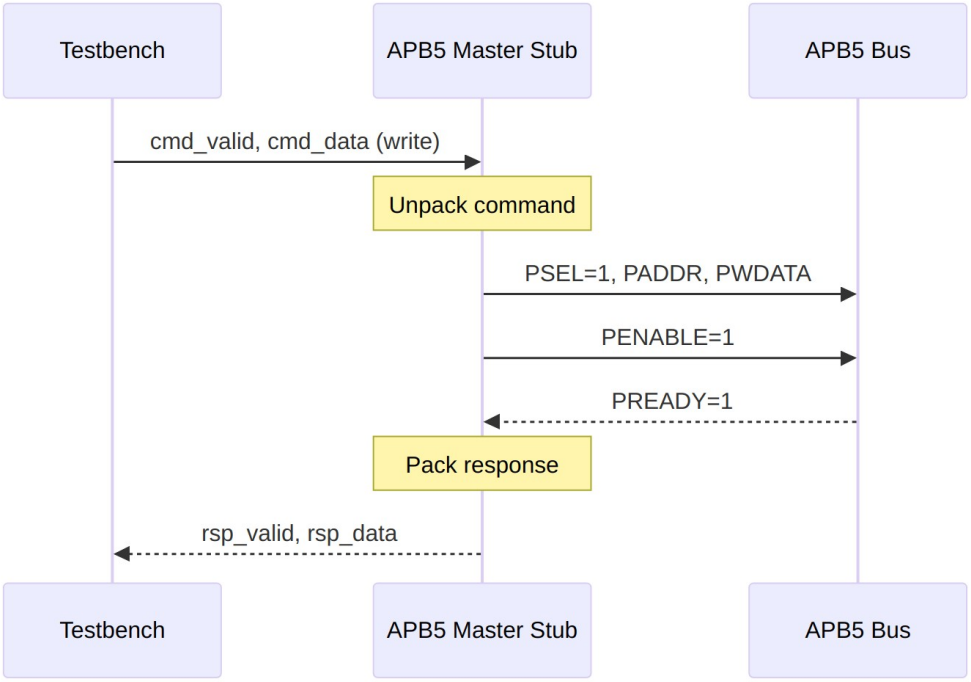


Response Packet Structure

Bit Positions:

```
rsp_data = {last, first, pslverr, pwakeup, prdata, pruser, pbuser}
```

4.5 Transaction Flow



Write Transaction

4.5.1 Timing

Timing diagram pending. The signals and sequence this scenario exercises:

- pclk
- cmd_valid, cmd_ready, cmd_data
- APB signals (PSEL, PENABLE, PADDR, PWDATA, PREADY)
- rsp_valid, rsp_ready, rsp_data
- Packet-to-APB timing relationship

4.6 Usage Example

```
apb5_master_stub #(
    .CMD_DEPTH      (6),
    .RSP_DEPTH      (6),
    .ADDR_WIDTH     (32),
    .DATA_WIDTH     (32),
    .AUSER_WIDTH    (4),
    .WUSER_WIDTH    (4),
```

```
    .RUSER_WIDTH    (4),
    .BUSER_WIDTH    (4),
    .ENABLE_PARITY  (0)
) u_apb5_master_stub (
    .pclk           (apb_clk),
    .presetn        (apb_rst_n),

    // APB5 master interface
    .m_apb_PSEL     (m_apb_psel),
    .m_apb_PENABLE  (m_apb_penable),
    .m_apb_PADDR    (m_apb_paddr),
    .m_apb_PWRITE   (m_apb_pwrite),
    .m_apb_PWDATA   (m_apb_pwdata),
    .m_apb_PSTRB    (m_apb_pstrb),
    .m_apb_PPROT    (m_apb_pprot),
    .m_apb_PAUSER   (m_apb_pauser),
    .m_apb_PWUSER   (m_apb_pwuser),
    .m_apb_PRDATA   (m_apb_prdata),
    .m_apb_PSLVERR  (m_apb_pslverr),
    .m_apb_PREADY   (m_apb_pready),
    .m_apb_PWAKEUP  (m_apb_pwakeup),
    .m_apb_PRUSER   (m_apb_pruser),
    .m_apb_PBUSER   (m_apb_pbuser),

    // Packed command interface
    .cmd_valid      (tb_cmd_valid),
    .cmd_ready      (tb_cmd_ready),
    .cmd_data       (tb_cmd_data),

    // Packed response interface
    .rsp_valid      (tb_rsp_valid),
    .rsp_ready      (tb_rsp_ready),
    .rsp_data       (tb_rsp_data),

    // Status
    .parity_error_rdata (),
    .parity_error_ctrl  (),
    .wakeup_pending (apb_wakeup)
);

// Build command packet.
// Field widths follow the module parameters; the literals below
assume the
// defaults, giving CMD_PACKET_WIDTH = 82.
assign tb_cmd_data = {
    1'b1,           // last     (1 bit)
    1'b1,           // first    (1 bit)
```

```
    1'b1,           // pwrite   (1 bit, write transaction)
    3'b000,         // pprot    (PROT_WIDTH  = 3)
    4'hF,           // pstrb    (STRB_WIDTH  = 4, all bytes)
    32'h1000_0000,  // paddr    (ADDR_WIDTH  = 32)
    32'hDEAD_BEEF,  // pwdata   (DATA_WIDTH  = 32)
    4'h0,           // pauser   (AUSER_WIDTH = 4)
    4'h0            // pwuser   (WUSER_WIDTH = 4)
};
```

For a parameterized testbench, build the packet from the parameters rather than from literals so it tracks any width change:

```
assign tb_cmd_data = {
    cmd_last, cmd_first, cmd_pwrite,
    cmd_pprot[PROT_WIDTH-1:0],
    cmd_pstrb[STRB_WIDTH-1:0],
    cmd_paddr[ADDR_WIDTH-1:0],
    cmd_pwdata[DATA_WIDTH-1:0],
    cmd_pauser[AUSER_WIDTH-1:0],
    cmd_pwuser[WUSER_WIDTH-1:0]
};
```

4.7 Design Notes

4.7.1 First/Last Markers

The first/last bits in packets enable: - Transaction boundary detection - Burst transaction support - Testbench synchronization

4.7.2 Internal Architecture

The stub instantiates the full `apb5_master` internally: - All protocol handling done by proven `apb5_master` - Stub only handles packet packing/unpacking - Maintains full APB5 compliance

---

4.8 Related Documentation

- **APB5 Master** - Core master module (wrapped by stub)
- **APB5 Slave Stub** - Corresponding slave stub

4.9 Navigation

- **← Back to APB5 Index**

- ← Back to RTLAmba Index
- ← Back to Main Documentation Index

5 APB5 Slave

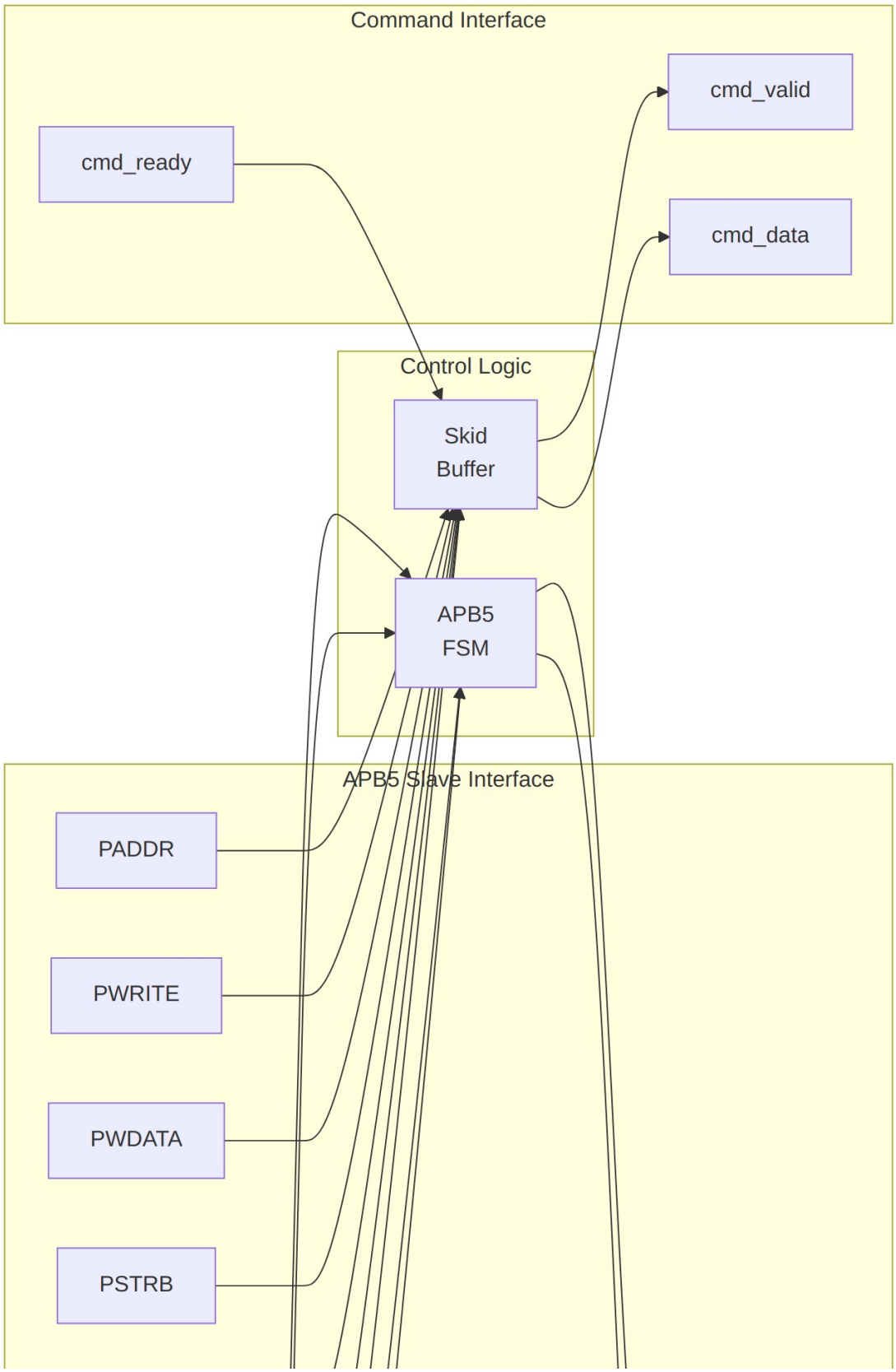
Module: `apb5_slave.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

5.1 Overview

The APB5 Slave module implements a complete AMBA APB5 slave interface with all APB5 extensions. It receives APB transactions from the bus and provides a command/response interface for backend logic, supporting user-defined signals, wake-up generation, and optional parity.

5.1.1 Key Features

- Full AMBA APB5 protocol compliance
- Receives PAUSER/PWUSER from master
- Generates PRUSER/PBUSER responses
- PWAKEUP output for wake-up signaling
- Optional parity support for data integrity
- Command/response FIFO buffering
- Configurable buffer depth


Command Interface
cmd_valid
cmd_ready
cmd_data
Control Logic
Skid
Buffer
APB5
FSM
APB5 Slave Interface
PADDR
PWRITE
PWDATA
PSTRB

5.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address/request user signal width
WUSER_WIDTH	int	4	Write data user signal width
RUSER_WIDTH	int	4	Read data user signal width
BUSER_WIDTH	int	4	Response user signal width
DEPTH	int	2	Skid-buffer depth in entries; must be one of {2, 4, 6, 8}
ENABLE_PARITY	bit	0	Enable parity generation and checking
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)

`DEPTH` sets both the command and the response `gaxi_skid_buffer` depth. It is a literal entry count, not a log2 exponent.

5.3 Ports

5.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB active-low

Port	Width	Direction	Description
			reset

5.3.2 APB5 Slave Interface

Port	Width	Direction	Description
s_apb_PSEL	1	Input	APB select signal
s_apb_PENABLE	1	Input	APB enable signal
s_apb_PREADY	1	Output	Slave ready response
s_apb_PADDR	ADDR_WIDTH	Input	APB address
s_apb_PWRITE	1	Input	Write/read indicator
s_apb_PWDATA	DATA_WIDTH	Input	Write data
s_apb_PSTRB	STRB_WIDTH	Input	Write byte strobes
s_apb_PPROT	PROT_WIDTH	Input	Protection attributes
s_apb_PAUSER	AUSER_WIDTH	Input	User-defined request attributes
s_apb_PWUSER	WUSER_WIDTH	Input	User-defined write data attributes
s_apb_PRDATA	DATA_WIDTH	Output	Read data to master
s_apb_PSLVERR	1	Output	Slave error response
s_apb_PWAKEUP	1	Output	Wake-up signal to master
s_apb_PRUSER	RUSER_WIDTH	Output	User-defined read data attributes
s_apb_PBU	BUSER_WIDT	Output	User-defined response

Port	Width	Direction	Description
SER	H		attributes

5.3.3 Parity Signals (Optional)

Port	Width	Direction	Description
s_apb_PWDATAPARITY	STRB_WIDTH	Input	Write data parity from master
s_apb_PADDRPARITY	1	Input	Address parity from master
s_apb_PCTRLPARITY	1	Input	Control signals parity from master
s_apb_PRDATAPARITY	STRB_WIDTH	Output	Read data parity to master
s_apb_PREADYPARITY	1	Output	PREADY parity to master
s_apb_PSLVERRPARITY	1	Output	PSLVERR parity to master

5.3.4 Command Interface (to backend)

Port	Width	Direction	Description
cmd_valid	1	Output	Command valid to backend
cmd_ready	1	Input	Backend ready
cmd_pwrite	1	Output	Command write/read
cmd_paddr	ADDR_WIDTH	Output	Command address
cmd_pwdata	DATA_WIDTH	Output	Command write data
cmd_pstrb	STRB_WIDTH	Output	Command

Port	Width	Direction	Description
			write strobes
cmd_pprot	PROT_WIDTH	Output	Command protection
cmd_pauser	AUSER_WIDTH	Output	Command address user
cmd_pwuser	WUSER_WIDTH	Output	Command write user

5.3.5 Response Interface (from backend)

Port	Width	Direction	Description
rsp_valid	1	Input	Response valid from backend
rsp_ready	1	Output	Slave ready for response
rsp_prdata	DATA_WIDTH	Input	Response read data
rsp_pslverr	1	Input	Response error status
rsp_pruser	RUSER_WIDTH	Input	Response read user
rsp_pbuser	BUSER_WIDTH	Input	Response user

5.3.6 Wake-up Control

Port	Width	Direction	Description
wakeup_request	1	Input	Wake-up request from backend; registered onto `s_apb_PWAKEUP`

5.3.7 Status Outputs

Port	Width	Direction	Description
parity_error_wdata	1	Output	Write-data parity mismatch (tied to 0 when ENABLE_PARITY=0)
parity_erro	1	Output	Address or control parity

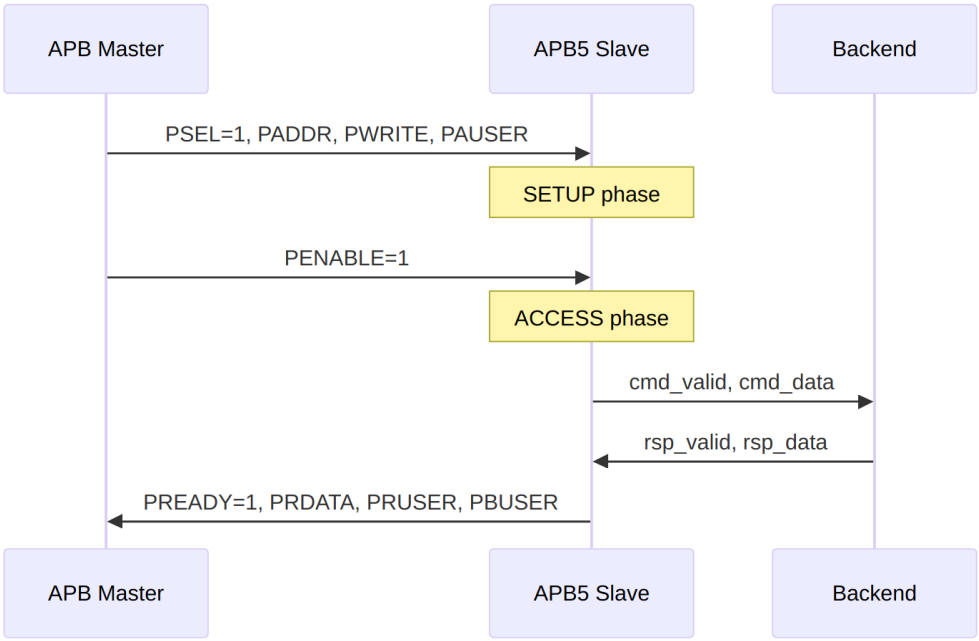
Port	Width	Direction	Description
r_ctrl			mismatch (tied to 0 when ENABLE_PARITY=0)

5.4 Functionality

5.4.1 APB5 Slave Protocol

The slave responds to APB transactions with proper timing:

1. **SETUP Phase**: PSEL=1, PENABLE=0 - Capture address and control
2. **ACCESS Phase**: PSEL=1, PENABLE=1 - Assert PREADY when response ready



Response Timing

5.5 Timing Diagrams

5.5.1 Write Transaction with Wait States

Timing diagram pending. The signals and sequence this scenario exercises:

- PCLK

- PSEL
- PENABLE
- PADDR
- PWRITE (high)
- PWDATA
- PAUSER, PWUSER
- PREADY (with wait states)
- PSLVERR
- PBUSER

5.5.2 Read Transaction

Timing diagram pending. The signals and sequence this scenario exercises:

- PCLK
- PSEL
- PENABLE
- PADDR
- PWRITE (low)
- PAUSER
- PREADY
- PRDATA
- PRUSER, PBUSER

5.5.3 Wake-up Signaling

Timing diagram pending. The signals and sequence this scenario exercises:

- PCLK
- wakeup_request (from backend)
- PWAKEUP (to master)
- Timing relationship

5.6 Usage Example

```
apb5_slave #(
    .ADDR_WIDTH     (32),
```

```
    .DATA_WIDTH     (32),
    .AUSER_WIDTH    (4),
    .WUSER_WIDTH    (4),
    .RUSER_WIDTH    (4),
    .BUSER_WIDTH    (4),
    .DEPTH          (2),
    .ENABLE_PARITY  (0)
) u_apb5_slave (
    .pclk           (apb_clk),
    .presetn        (apb_rst_n),

    // APB5 slave interface
    .s_apb_PSEL     (s_apb_psel),
    .s_apb_PENABLE  (s_apb_penable),
    .s_apb_PREADY   (s_apb_pready),
    .s_apb_PADDR    (s_apb_paddr),
    .s_apb_PWRITE   (s_apb_pwrite),
    .s_apb_PWDATA   (s_apb_pwdata),
    .s_apb_PSTRB    (s_apb_pstrb),
    .s_apb_PPROT    (s_apb_pprot),
    .s_apb_PAUSER   (s_apb_pauser),
    .s_apb_PWUSER   (s_apb_pwuser),
    .s_apb_PRDATA   (s_apb_prdata),
    .s_apb_PSLVERR  (s_apb_pslverr),
    .s_apb_PWAKEUP  (s_apb_pwakeup),
    .s_apb_PRUSER   (s_apb_pruser),
    .s_apb_PBUSER   (s_apb_pbuser),

    // Backend command interface
    .cmd_valid      (backend_cmd_valid),
    .cmd_ready      (backend_cmd_ready),
    .cmd_pwrite     (backend_cmd_write),
    .cmd_paddr      (backend_cmd_addr),
    .cmd_pwdata     (backend_cmd_wdata),
    .cmd_pstrb      (backend_cmd_strb),
    .cmd_pprot      (backend_cmd_prot),
    .cmd_pauser     (backend_cmd_auser),
    .cmd_pwuser     (backend_cmd_wuser),

    // Backend response interface
    .rsp_valid      (backend_rsp_valid),
    .rsp_ready      (backend_rsp_ready),
    .rsp_prdata     (backend_rsp_rdata),
    .rsp_pslverr    (backend_rsp_error),
    .rsp_pruser     (backend_rsp_ruser),
    .rsp_pbuser     (backend_rsp_buser),
```

```
    // Wake-up
    .wakeup_request (backend_wakeup),

    // Parity interface (unused here because ENABLE_PARITY=0)
    .s_apb_PWDATAPARITY  ('0),
    .s_apb_PADDRPARITY   (1'b0),
    .s_apb_PCTRLPARITY   (1'b0),
    .s_apb_PRDATAPARITY  (),
    .s_apb_PREADYPARITY  (),
    .s_apb_PSLVERRPARITY (),
    .parity_error_wdata  (),
    .parity_error_ctrl   ()
);
```

5.7 Design Notes

5.7.1 Backpressure Handling

- If backend cannot accept commands (`cmd_ready=0`), slave inserts wait states
- PREADY held low until backend accepts command and provides response

5.7.2 User Signal Propagation

- PAUSER/PWUSER captured during SETUP phase, forwarded to backend
- PRUSER/PBUSER from backend driven during ACCESS phase response

5.7.3 Wake-up Generation

`wakeup_request` is registered once before it drives `s_apb_PWAKEUP`, giving a single-cycle delay from request to assertion. The slave does not qualify PWAKEUP with bus state – it simply mirrors the (registered) backend request.

5.7.4 Parity Implementation

When `ENABLE_PARITY=1` the slave checks the parity the master supplies and generates parity for its own outputs:

Parity signal	Direction	Covers	Granularity
s_apb_PWDATAPARITY[i]	Checked	PWDATA byte lane `i`	One bit per byte (STRB_WIDTH bits total)
s_apb_PADDRPARITY	Checked	Whole	One bit for the

Parity signal	Direction	Covers	Granularity
		PADDR	entire address
s_apb_PCTRLPARITY	Checked	{PWRITE, PSTRB, PPROT} concatenated	One bit for the whole control group
s_apb_PRDATAPARITY[i]	Generated	PRDATA byte lane `i`	One bit per byte
s_apb_PREADYPARITY	Generated	PREADY	One bit
s_apb_PSLVERRPARITY	Generated	PSLVERR	One bit

Each parity bit is the XOR reduction of the covered signals, so it is 1 when the covered field contains an odd number of ones (an even-parity encoding). Per-byte data parity detects one bit error in each byte independently; the single address and control bits detect only an odd number of errors across the whole group.

`parity_error_wdata` and `parity_error_ctrl` are combinational and qualified by `s_apb_PSEL && s_apb_PENABLE`; they read 0 outside the ACCESS phase and are hard-tied to 0 when `ENABLE_PARITY=0`. The slave reports mismatches but does not itself convert them into PSLVERR – that policy is left to the integrator.

5.8 Related Documentation

- **APB5 Master** - APB5 master interface
- **APB5 Slave CG** - Clock-gated variant
- **APB5 Slave CDC** - Clock domain crossing variant
- **APB4 Slave** - APB4 version for comparison

5.9 Navigation

- **← Back to APB5 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

6 APB5 Slave (Clock Domain Crossing)

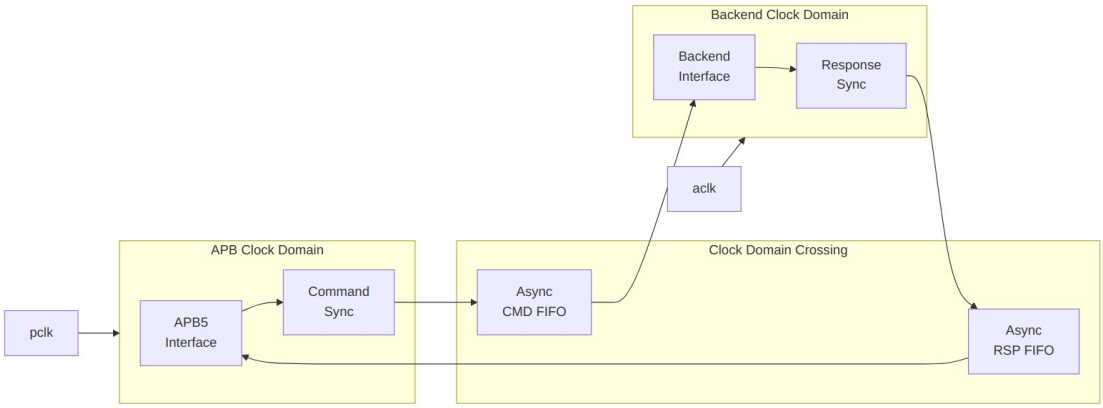
Module: `apb5_slave_cdc.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

6.1 Overview

The APB5 Slave CDC module provides clock domain crossing capability between an APB5 bus clock domain and a backend clock domain. It safely transfers APB5 transactions across asynchronous clock boundaries.

6.1.1 Key Features

- Full APB5 protocol support with CDC
- Asynchronous FIFO-based clock domain crossing (Gray-coded pointers)
- All APB5 user signals (PAUSER, PWUSER, PRUSER, PBUSER)
- Wake-up request crossing via a dedicated level synchronizer
- Single `DEPTH` parameter sizes both directions
- Metastability protection with 2-flop pointer synchronizers



Module Architecture

6.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width

Parameter	Type	Default	Description
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address user signal width
WUSER_WIDTH	int	4	Write user signal width
RUSER_WIDTH	int	4	Read user signal width
BUSER_WIDTH	int	4	Response user signal width
DEPTH	int	2	Skid-buffer depth of the wrapped `apb5_slave`; one of {2, 4, 6, 8}
ENABLE_PARITY	bit	0	Enable parity generation and checking
USE_2_PHASE_CDC	bit	1	Deprecated and ignored – retained for source compatibility

There is no `CMD_DEPTH`, `RSP_DEPTH` or `SYNC_STAGES` parameter on this module. The two asynchronous FIFOs are sized internally from `DEPTH`:

```
localparam int CDC_FIFO_DEPTH = (DEPTH < 4) ? 4 : DEPTH;
```

so the CDC FIFOs are never shallower than 4 entries regardless of `DEPTH`. The synchronizer depth is fixed at 2 flops (`N_FLOP_CROSS(2)` on both `gaxi_fifo_async` instances) and is not exposed as a parameter. A design needing 3-flop synchronization for an extreme clock ratio must change the instantiation in the RTL.

6.3 Ports

6.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB bus clock
presetn	1	Input	APB reset (active low)

Port	Width	Direction	Description
aclk	1	Input	Backend/user clock
aresetn	1	Input	Backend/user reset (active low)

The backend clock and reset are named `aclk` and `aresetn`, matching the rest of the AMBA library. There are no `bclk`/`bresetn` ports.

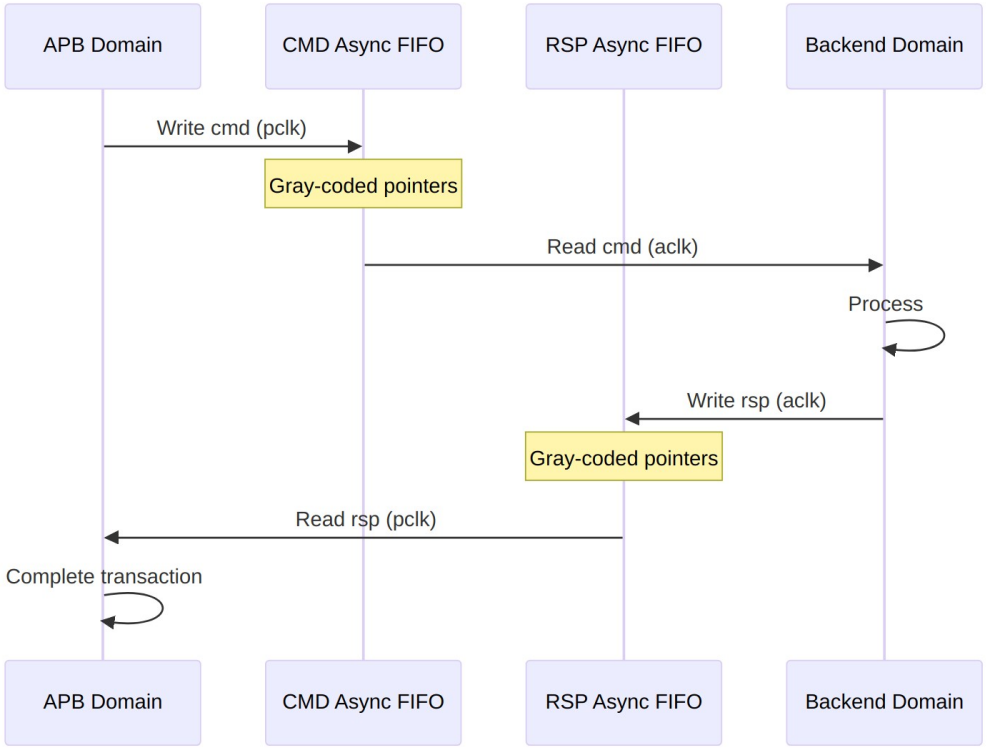
6.3.2 APB5 Slave Interface

Same as apb5_slave - operates in `pclk` domain, including the optional parity signals.

6.3.3 Backend Interface

Same command/response interface as apb5_slave - operates in the `aclk` domain. `wakeup_request`, `parity_error_wdata` and `parity_error_ctrl` are also present.

6.4 Clock Domain Crossing



CDC Mechanism

6.4.1 Timing Considerations

Timing diagram pending. The signals and sequence this scenario exercises:

- pclk
- aclk (different frequency)
- APB transaction signals
- FIFO write/read pointers
- Backend transaction signals
- CDC latency

6.5 Usage Example

```
apb5_slave_cdc #(
    .ADDR_WIDTH     (32),
    .DATA_WIDTH     (32),
    .AUSER_WIDTH    (4),
```

```
    .WUSER_WIDTH    (4),
    .RUSER_WIDTH    (4),
    .BUSER_WIDTH    (4),
    .DEPTH          (2),
    .ENABLE_PARITY  (0)
) u_apb5_slave_cdc (
    // APB clock domain
    .pclk           (apb_clk),
    .presetn        (apb_rst_n),

    // Backend clock domain
    .aclk           (backend_clk),
    .aresetn        (backend_rst_n),

    // APB5 slave interface (pclk domain)
    .s_apb_PSEL     (s_apb_psel),
    .s_apb_PENABLE  (s_apb_penable),
    // ... other APB signals

    // Backend interface (aclk domain)
    .cmd_valid      (backend_cmd_valid),
    .cmd_ready      (backend_cmd_ready),
    // ... other backend signals

    // Wake-up request (aclk domain, synchronized internally to pclk)
    .wakeup_request (backend_wakeup)
);
```

6.6 Design Notes

6.6.1 CDC Mechanism

Both directions cross through `gaxi_fifo_async` instances with Gray-coded pointers and a fixed 2-flop synchronizer (`N_FLOP_CROSS(2)`) on each pointer crossing. The command FIFO is written in `pclk` and read in `aclk`; the response FIFO is written in `aclk` and read in `pclk`.

The `wakeup_request` input is the one signal that does not go through a FIFO: it crosses from `aclk` into `pclk` through a `cdc_synchronizer` before reaching the wrapped `apb5_slave`. Because it is a level, not a pulse, this is safe – but it means `wakeup_request` must be held asserted long enough to be sampled in the `pclk` domain (at least two `pclk` periods).

6.6.2 FIFO Depth Sizing

- `DEPTH` sizes the `apb5_slave` skid buffers; the CDC FIFOs are sized separately as `max(DEPTH, 4)`, so the crossing itself is never shallower than 4 entries even at the default `DEPTH=2`
- Deeper FIFOs buy tolerance for a slow backend or a bursty APB master; they do not reduce the per-transfer crossing latency
- Because APB is single-outstanding, depth beyond a handful of entries has little effect on throughput for this module

6.6.3 Reset Synchronization

`presetn` and `aresetn` are independent reset domains and either may be asserted alone. `gaxi_fifo_async` resets each domain's own pointer *and* that domain's crossed copy of the remote pointer from the local reset, so a one-sided reset leaves that side self-consistent (both pointers zero, meaning empty) instead of fabricating or swallowing a transfer. Pointers are absolute positions rather than toggle parity, which is what makes the one-sided case safe.

The practical consequence: resetting only the backend (`aresetn`) discards in-flight commands from that side's point of view while the APB side stays up. A transfer already accepted on the APB side but not yet read out will not be delivered, so the APB master may see a transaction time out. Integrators who pulse only one reset should quiesce the bus first.

Neither reset is internally synchronized to the other domain's clock; each is expected to be already synchronized (or asynchronously asserted and synchronously deasserted) in its own domain by the integrator.

6.6.4 Latency

- Crossing latency is the async-FIFO write-to-read pointer synchronization: on the order of 2-3 destination-clock cycles per direction, plus the source-clock cycle that performs the write
- A full APB transfer therefore costs the command crossing plus the response crossing before PREADY can assert
- Additional latency if the FIFOs are full (backpressure) or the backend is slow

---

6.7 Related Documentation

- **APB5 Slave** - Base slave without CDC
- **APB5 Slave CDC CG** - CDC with clock gating
- **GAXI Async FIFO** - The async FIFO used for both crossings

- **Clock Domain Crossing** - CDC design patterns and reset behavior

---

6.8 Navigation

- **← Back to APB5 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

7 APB5 Slave (CDC + Clock-Gated)

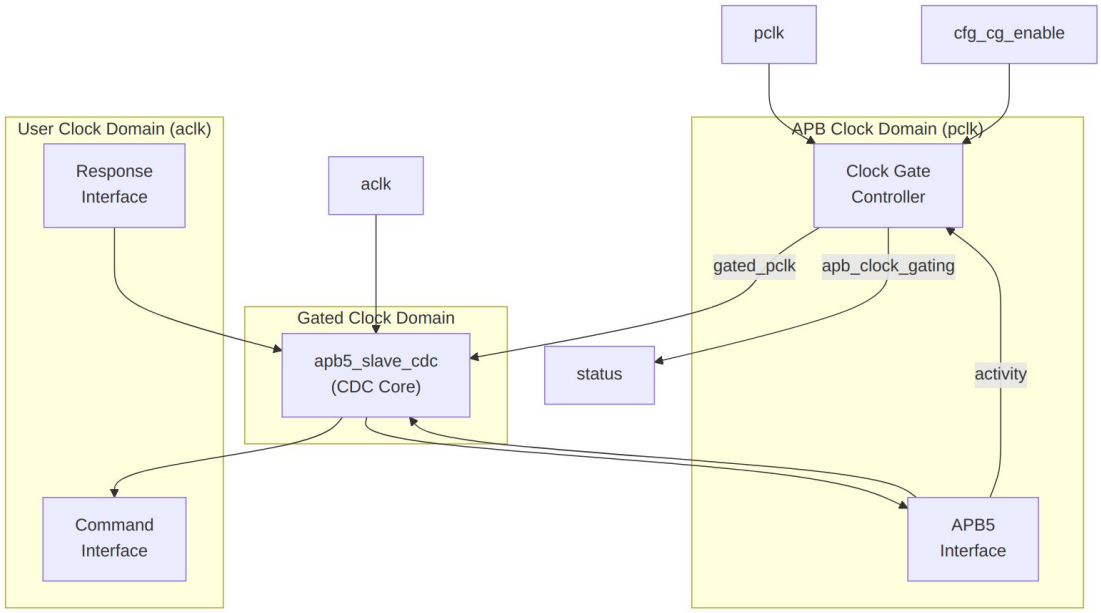
Module: `apb5_slave_cdc_cg.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

7.1 Overview

The APB5 Slave CDC + Clock-Gated module combines clock domain crossing with clock gating for maximum power efficiency. It wraps `apb5_slave_cdc` with clock gating control to reduce power consumption during idle periods while maintaining safe operation across asynchronous clock boundaries.

7.1.1 Key Features

- Full APB5 protocol support with all extensions
- Asynchronous clock domain crossing (APB to backend)
- Clock gating for power reduction during idle
- All APB5 user signals (PAUSER, PWUSER, PRUSER, PBUSER)
- PWAKEUP signal handling across domains
- Optional parity support for data integrity
- Automatic wake-up on transaction activity



Module Architecture

7.2 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address user signal width
WUSER_WIDTH	int	4	Write user signal width
RUSER_WIDTH	int	4	Read user signal width
BUSER_WIDTH	int	4	Response user signal width
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width (calculated)
DEPTH	int	2	Skid-buffer depth of the wrapped slave; CDC FIFOs use `max(DEPTH, 4)`
ENABLE_PARITY	bit	0	Enable parity generation

Parameter	Type	Default	Description
			and checking
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter (max idle = 2^N-1 cycles)
USE_2_PHASE_CDC	bit	1	Deprecated and ignored – retained for source compatibility

As with apb5_slave_cdc, there is no `SYNC_STAGES` parameter: pointer synchronization is fixed at 2 flops inside the async FIFOs.

7.3 Ports

7.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB bus clock
presetn	1	Input	APB reset (active low)
aclk	1	Input	User/backend clock
aresetn	1	Input	User reset (active low)

7.3.2 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Enable clock gating
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating

7.3.3 APB5 Slave Interface

Same as apb5_slave - operates in `pclk` domain.

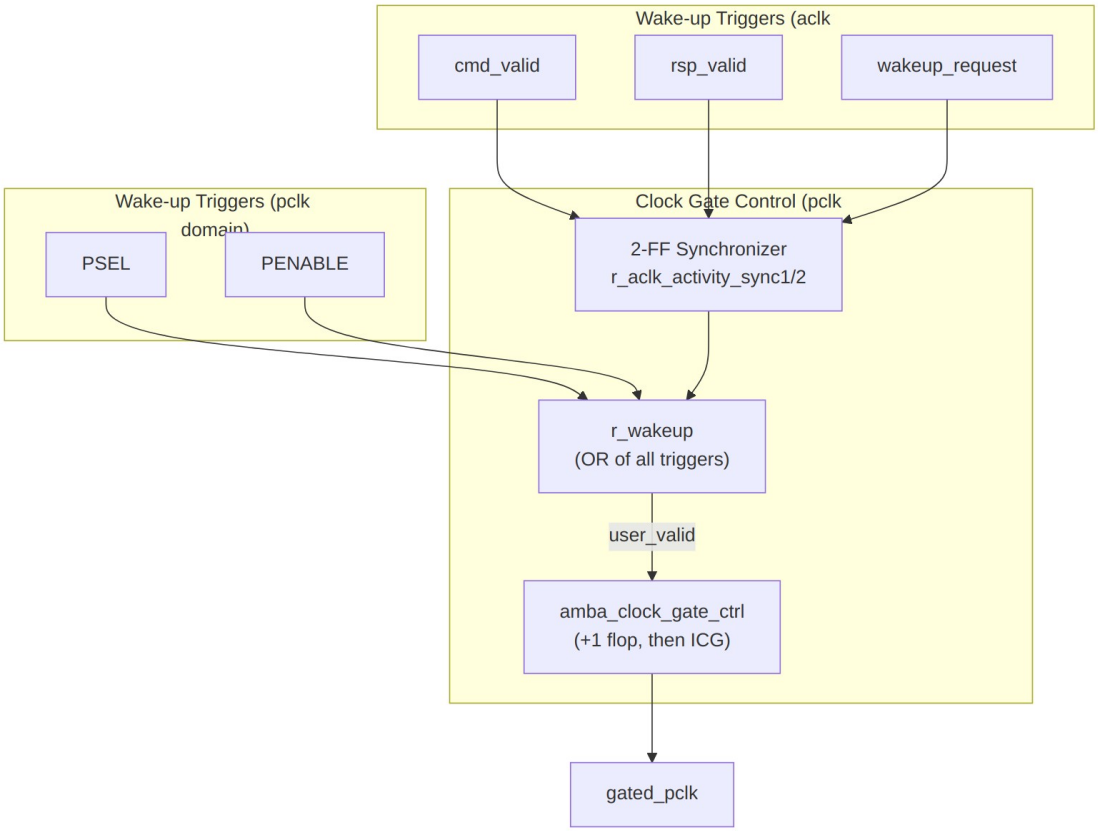
7.3.4 Backend Interface

Same command/response interface as apb5_slave_cdc - operates in `aclk` domain.

7.3.5 Status Outputs

Port	Width	Direction	Description
apb_clock_gating	1	Output	Indicates clock is currently gated
parity_error_wdata	1	Output	Write data parity error detected
parity_error_ctrl	1	Output	Control signal parity error

7.4 Clock Gating and CDC Interaction



Wake-up Logic

The `aclk`-domain activity terms are ORed together and passed through a two-flop synchronizer before being combined with the `pclk`-domain terms. Without that synchronizer these signals would cross domains unsynchronized.

7.4.1 Timing Considerations

Timing diagram pending. The signals and sequence this scenario exercises:

- pclk (ungated)
- gated_pclk
- aclk (different frequency)
- s_apb_PSEL (wake trigger)
- apb_clock_gating indicator
- Transaction flow across CDC with gating
- Wake-up latency from PSEL to clock active

7.5 Usage Example

```
apb5_slave_cdc_cg #(
    .ADDR_WIDTH         (32),
    .DATA_WIDTH         (32),
    .AUSER_WIDTH        (4),
    .WUSER_WIDTH        (4),
    .RUSER_WIDTH        (4),
    .BUSER_WIDTH        (4),
    .DEPTH              (2),
    .ENABLE_PARITY      (0),
    .CG_IDLE_COUNT_WIDTH(4)
) u_apb5_slave_cdc_cg (
    // APB clock domain
    .pclk               (apb_clk),
    .presetn            (apb_rst_n),

    // User clock domain
    .aclk               (user_clk),
    .aresetn            (user_rst_n),

    // Clock gating
    .cfg_cg_enable      (1'b1),
    .cfg_cg_idle_count  (4'd8),
    .apb_clock_gating   (slave_clk_gated),

    // APB5 slave interface (pclk domain)
    .s_apb_PSEL         (s_apb_psel),
    .s_apb_PENABLE      (s_apb_penable),
    .s_apb_PREADY       (s_apb_pready),
    // ... other APB signals

    // Backend interface (aclk domain)
    .cmd_valid          (backend_cmd_valid),
    .cmd_ready          (backend_cmd_ready),
    // ... other command signals

    .rsp_valid          (backend_rsp_valid),
    .rsp_ready          (backend_rsp_ready),
    // ... other response signals

    // Wake-up control (aclk domain)
    .wakeup_request     (backend_wakeup)
);
```

7.6 Design Notes

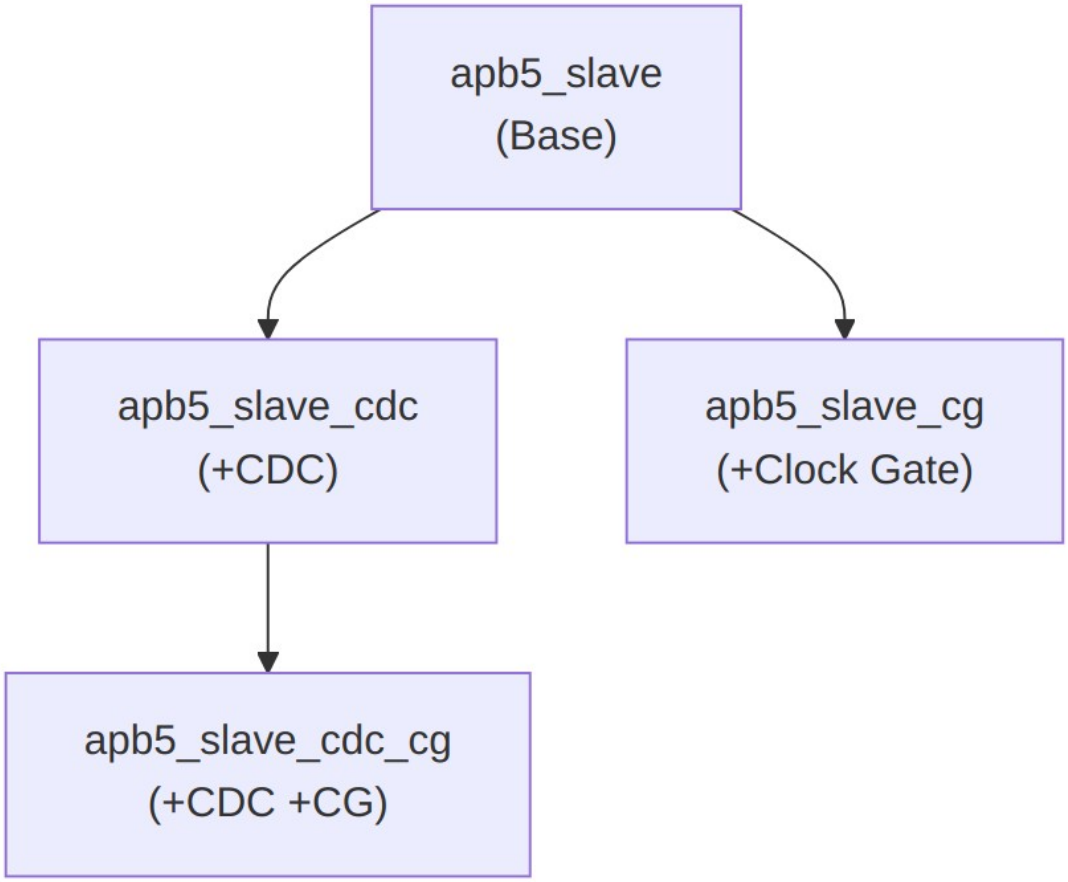
7.6.1 Power and Latency Trade-offs

Wake-up is **registered, not combinational**. Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream) before reaching the ICG enable, which is combinational. A pclk-domain trigger (PSEL or PENABLE) passes through this wrapper's `r_wakeup` and then through the flop inside `amba_clock_gate_ctrl`, so APB5 is a two-stage family and the first usable gated `pclk` rising edge arrives **3 ungated `pclk` cycles** after the trigger. An aclk-domain trigger (`cmd_valid`, `rsp_valid`, `wakeup_request`) first crosses a 2-flop synchronizer into `pclk`, adding about two more `pclk` cycles.

Configuration	Power Savings	Wake-up Latency from PSEL
CG disabled	None	0 cycles (clock always running)
CG idle=4	Moderate	2 register stages; first usable edge 3 ungated pclk cycles
CG idle=16	Good	2 register stages; first usable edge 3 ungated pclk cycles

The wake-up latency does not depend on `cfg_cg_idle_count`; the idle count only controls how long the block waits before gating. Latency is absorbed as APB wait states, since the slave holds PREADY low until it is running again.

Deliberately keeping the wake-up path registered avoids driving a clock-gate enable from a combinational function of a bus input. The gating element itself is an ICG cell instantiated by `clock_gate_ctrl`, which is glitch-free by construction. ICG cells are an ASIC library primitive; FPGA targets should use a clock-enable approach instead.



Combined Feature Hierarchy

7.6.2 Reset Considerations

- APB domain reset (`presetn`) controls the APB interface, the wake-up synchronizer and the clock gate
- User domain reset (`aresetn`) controls the backend side of the CDC FIFOs
- Each reset must already be synchronized (or asynchronously asserted and synchronously deasserted) in its own domain by the integrator; this module does not synchronize either reset into the other domain
- The async FIFOs tolerate a one-sided reset without corrupting the pointer state – see apb5_slave_cdc for the mechanism and for the in-flight transaction caveat

7.7 Related Documentation

- **APB5 Slave** - Base slave module
- **APB5 Slave CDC** - CDC variant without clock gating
- **APB5 Slave CG** - Clock gating without CDC

7.8 Navigation

- **← Back to APB5 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

8 APB5 Slave (Clock-Gated)

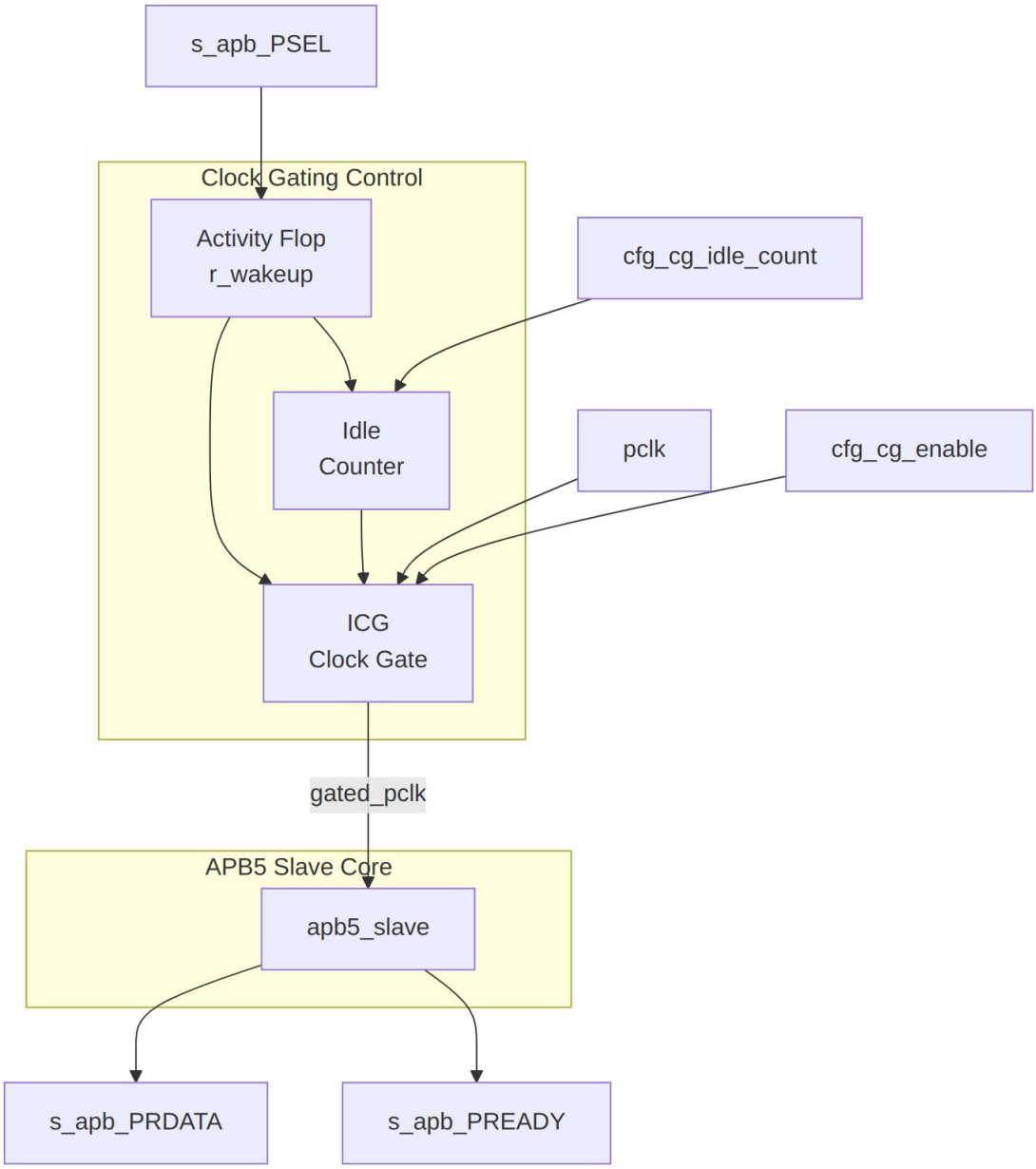
Module: `apb5_slave_cg.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

8.1 Overview

Clock-gated variant of the APB5 Slave module. Wraps the base `apb5_slave` with clock gating control logic to reduce dynamic power consumption when no transactions are active.

8.1.1 Key Features

- All APB5 Slave features (see apb5_slave)
- Automatic clock gating during idle periods
- Registered wake-up on PSEL assertion, through a glitch-free ICG cell
- Configurable idle threshold
- Gating status output



Module Architecture

8.2 Additional Parameters

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter

All other parameters inherited from apb5_slave.

8.3 Additional Ports

8.3.1 Clock Gating Configuration

Port	Width	Direction	Description
cfg_cg_enable	1	Input	Enable clock gating
cfg_cg_idle_count	CG_IDLE_COUNT_WIDTH	Input	Idle cycles before gating

8.3.2 Clock Gating Status

Port	Width	Direction	Description
apb_clock_gating	1	Output	High while the internal clock is gated off

There is no cumulative gated-cycle counter port on this module. If a gated-cycle total is needed, count `apb_clock_gating` in the integrating logic on the ungated `pclk`.

All ports of apb5_slave – including the parity signals, `wakeup_request`, `parity_error_wdata` and `parity_error_ctrl` – are present unchanged and pass straight through to the wrapped core.

8.4 Clock Gating Behavior

8.4.1 Wake-up Trigger

The wrapper keeps the clock running whenever any of the following is high:

```
s_apb_PSEL || s_apb_PENABLE || cmd_valid || rsp_valid ||
wakeup_request
```

That term is registered into `r_wakeup` on the ungated `pclk`, and `amba_clock_gate_ctrl` registers it once more before it reaches the gating condition. Activity is registered once (AXI4, AXI5, AXI4-Lite, AXI4-Stream) or twice (APB, APB5, AXI5-Stream) before reaching the ICG enable, which is combinational. APB5 is a **two-stage** family, so the first gated-clock rising edge available to the wrapped `apb5_slave` arrives **3 ungated `pclk` cycles** after activity asserts. Because the slave signals not-ready by holding PREADY low, those cycles simply appear to the master as APB wait states.

The gating element is an ICG (integrated clock gating) cell instantiated by `clock_gate_ctrl`, so enable transitions are glitch-free. ICG cells are an ASIC library primitive; on FPGA targets use a clock-enable approach instead.

Gating engages `cfg_cg_idle_count + 1` ungated `pclk` cycles after the internal wakeup deasserts, which is `cfg_cg_idle_count + 3` cycles after the last bus activity, because APB5 adds two register stages ahead of the ICG enable.

> **Timing diagram pending.** The signals and sequence this scenario exercises:

- pclk
- gated_pclk
- s_apb_PSEL
- apb_clock_gating
- Wake-up latency (two register stages; first usable gated edge 3 ungated pclk cycles after activity)

8.5 Usage Example

```
apb5_slave_cg #(
    .ADDR_WIDTH         (32),
    .DATA_WIDTH         (32),
    .CG_IDLE_COUNT_WIDTH(4)
) u_apb5_slave_cg (
    .pclk               (apb_clk),
    .presetn            (apb_rst_n),

    // Clock gating
    .cfg_cg_enable      (1'b1),
    .cfg_cg_idle_count  (4'd4),
    .apb_clock_gating   (slave_clk_gated),

    // APB5 interface (same as apb5_slave)
    // ...
);
```

8.6 Related Documentation

- **APB5 Slave** - Base module documentation
- **APB5 Master CG** - Clock-gated master

8.7 Navigation

- **← Back to APB5 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

9 APB5 Slave Stub

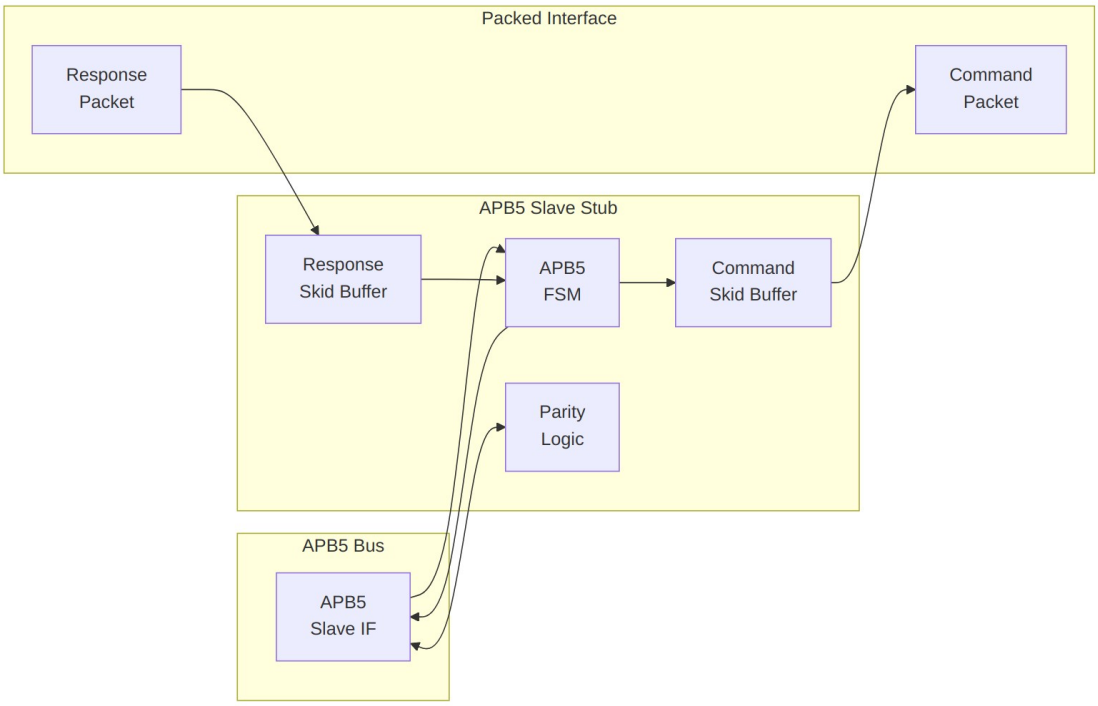
Module: `apb5_slave_stub.sv` **Location:** `rtl/amba/apb5/` **Status:** Production Ready

9.1 Overview

The APB5 Slave Stub provides a simplified packed-data interface for responding to APB5 transactions. It implements a complete APB5 slave with packet-based command/response interfaces, making it ideal for testbenches and simple backend implementations.

9.1.1 Key Features

- Packed command/response packet interface
- Complete APB5 slave FSM implementation
- All APB5 extensions (PAUSER, PWUSER, PRUSER, PBUSER)
- PWAKEUP signal generation to master
- Optional parity support with error detection
- Skid buffers for command and response paths



Module Architecture

9.2 Parameters

Parameter	Type	Default	Description
DEPTH	int	4	Skid-buffer depth in entries; one of {2, 4, 6, 8}
ADDR_WIDTH	int	32	APB address bus width
DATA_WIDTH	int	32	APB data bus width
PROT_WIDTH	int	3	Protection signal width
AUSER_WIDTH	int	4	Address user signal width
WUSER_WIDTH	int	4	Write user signal width

Parameter	Type	Default	Description
RUSER_WIDTH	int	4	Read user signal width
BUSER_WIDTH	int	4	Response user signal width
ENABLE_PARITY	bit	0	Enable parity signals
STRB_WIDTH	int	DATA_WIDTH/8	Write strobe width
CMD_PACKET_WIDTH	int	(calculated)	Total command packet width
RESP_PACKET_WIDTH	int	(calculated)	Total response packet width

`CMD_PACKET_WIDTH` and `RESP_PACKET_WIDTH` are computed from the width parameters and must not be overridden:

```
CMD_PACKET_WIDTH  = ADDR_WIDTH + DATA_WIDTH + STRB_WIDTH + PROT_WIDTH
                  + AUSER_WIDTH + WUSER_WIDTH + 1
                    // the single bit is pwrite

RESP_PACKET_WIDTH = DATA_WIDTH + RUSER_WIDTH + BUSER_WIDTH + 1
                    // the single bit is pslverr
```

With all defaults this gives `CMD_PACKET_WIDTH = 80` and `RESP_PACKET_WIDTH = 41`.

Unlike apb5_master_stub, the slave stub packets carry no `first`/`last` markers and no `pwakeup` bit, so the two stubs' packet widths are not interchangeable.

9.3 Ports

9.3.1 Clock and Reset

Port	Width	Direction	Description
pclk	1	Input	APB clock
presetn	1	Input	APB reset (active low)

9.3.2 APB5 Slave Interface

Port	Width	Direction	Description
s_apb_PSEL	1	Input	APB select signal
s_apb_PENABLE	1	Input	APB enable signal
s_apb_PADDR	ADDR_WIDTH	Input	APB address
s_apb_PWRITE	1	Input	Write/read indicator
s_apb_PWDATA	DATA_WIDTH	Input	Write data
s_apb_PSTRB	STRB_WIDTH	Input	Write byte strobes
s_apb_PPROT	PROT_WIDTH	Input	Protection attributes
s_apb_PAUSER	AUSER_WIDTH	Input	User request attributes
s_apb_PWUSER	WUSER_WIDTH	Input	User write attributes
s_apb_PRDATA	DATA_WIDTH	Output	Read data to master
s_apb_PSLVERR	1	Output	Slave error response
s_apb_PREADY	1	Output	Slave ready
s_apb_PWAKEUP	1	Output	Wake-up to master
s_apb_PRUSER	RUSER_WIDTH	Output	User read attributes
s_apb_PBUSER	BUSER_WIDTH	Output	User response attributes

9.3.3 Parity Signals (Optional)

Present unconditionally; meaningful only when `ENABLE_PARITY=1`. Semantics are identical to apb5_slave.

Port	Width	Direction	Description
s_apb_PWDATAPARITY	STRB_WIDTH	Input	Write data parity from master
s_apb_PADDRPARITY	1	Input	Address parity from master
s_apb_PCTRLPARITY	1	Input	Control signals parity from master
s_apb_PRDATAPARITY	STRB_WIDTH	Output	Read data parity to master
s_apb_PREADYPARITY	1	Output	PREADY parity to master
s_apb_PSLVERRPARITY	1	Output	PSLVERR parity to master

9.3.4 Command Packet Interface

Port	Width	Direction	Description
cmd_valid	1	Output	Command packet valid
cmd_ready	1	Input	Backend ready for command
cmd_data	CMD_PACKET_WIDTH	Output	Packed command data

9.3.5 Response Packet Interface

Port	Width	Direction	Description
rsp_valid	1	Input	Response packet valid
rsp_ready	1	Output	Ready for response
rsp_data	RESP_PACKET_WIDTH	Input	Packed response data

9.3.6 Control and Status

Port	Width	Direction	Description
wakeup_request	1	Input	Assert PWAKEUP to master
parity_error_wdata	1	Output	Write data parity error
parity_error_ctrl	1	Output	Control signal parity error

9.4 Packet Formats

9.4.1 Command Packet Structure

```
cmd_data = {pwrite, pprot, pstrb, paddr, pwdata, pauser, pwuser}
```

Field	Width	Description
pwrite	1	Write/read indicator
pprot	PROT_WIDTH	Protection attributes
pstrb	STRB_WIDTH	Write byte strobes
paddr	ADDR_WIDTH	Transaction address
pwdata	DATA_WIDTH	Write data
pauser	AUSER_WIDTH	Address user signal
pwuser	WUSER_WIDTH	Write user signal

9.4.2 Response Packet Structure

```
rsp_data = {pslverr, prdata, pruser, pbuser}
```

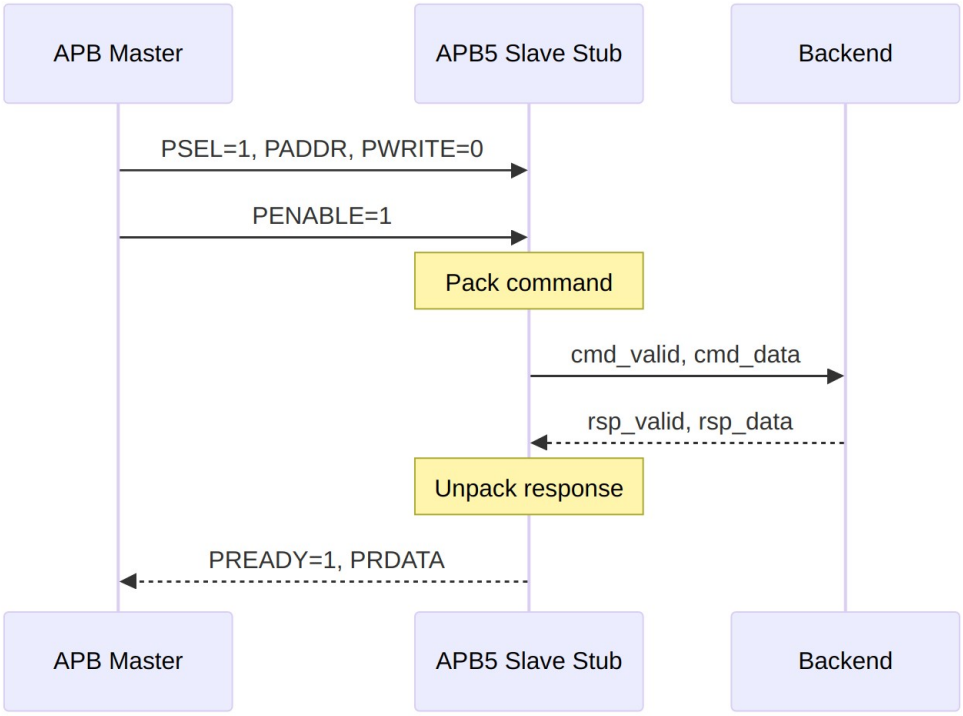
Field	Width	Description
pslverr	1	Error response
prdata	DATA_WIDTH	Read data
pruser	RUSER_WIDTH	Read user signal
pbuser	BUSER_WIDTH	Response user signal

```
stateDiagram-v2
    [*] --> IDLE

    IDLE --> XFER_DATA : PSEL & PENABLE & cmd_ready
    XFER_DATA --> IDLE : rsp_valid

    state IDLE {
        note right of IDLE : Wait for APB access
    }
    state XFER_DATA {
        note right of XFER_DATA : Wait for backend response
    }
```

9.4.3 State Descriptions

State	Description
IDLE	Waiting for APB transaction (PSEL & PENABLE)
XFER_DATA	Command sent, waiting for response from backend

9.5 Transaction Flow



Read Transaction

9.5.1 Timing

Timing diagram pending. The signals and sequence this scenario exercises:

- pclk
- APB signals (PSEL, PENABLE, PADDR, PWRITE, PREADY, PRDATA)
- cmd_valid, cmd_ready, cmd_data
- rsp_valid, rsp_ready, rsp_data
- State transitions IDLE -> XFER_DATA -> IDLE

9.6 Usage Example

```
apb5_slave_stub #(
    .DEPTH          (4),
    .ADDR_WIDTH     (32),
    .DATA_WIDTH     (32),
    .AUSER_WIDTH    (4),
    .WUSER_WIDTH    (4),
```

```
    .RUSER_WIDTH    (4),
    .BUSER_WIDTH    (4),
    .ENABLE_PARITY  (0)
) u_apb5_slave_stub (
    .pclk           (apb_clk),
    .presetn        (apb_rst_n),

    // APB5 slave interface
    .s_apb_PSEL     (s_apb_psel),
    .s_apb_PENABLE  (s_apb_penable),
    .s_apb_PADDR    (s_apb_paddr),
    .s_apb_PWRITE   (s_apb_pwrite),
    .s_apb_PWDATA   (s_apb_pwdata),
    .s_apb_PSTRB    (s_apb_pstrb),
    .s_apb_PPROT    (s_apb_pprot),
    .s_apb_PAUSER   (s_apb_pauser),
    .s_apb_PWUSER   (s_apb_pwuser),
    .s_apb_PRDATA   (s_apb_prdata),
    .s_apb_PSLVERR  (s_apb_pslverr),
    .s_apb_PREADY   (s_apb_pready),
    .s_apb_PWAKEUP  (s_apb_pwakeup),
    .s_apb_PRUSER   (s_apb_pruser),
    .s_apb_PBUSER   (s_apb_pbuser),

    // Packed command interface
    .cmd_valid      (be_cmd_valid),
    .cmd_ready      (be_cmd_ready),
    .cmd_data       (be_cmd_data),

    // Packed response interface
    .rsp_valid      (be_rsp_valid),
    .rsp_ready      (be_rsp_ready),
    .rsp_data       (be_rsp_data),

    // Wake-up control
    .wakeup_request (be_wakeup),

    // Parity interface (unused here because ENABLE_PARITY=0)
    .s_apb_PWDATAPARITY  ('0),
    .s_apb_PADDRPARITY   (1'b0),
    .s_apb_PCTRLPARITY   (1'b0),
    .s_apb_PRDATAPARITY  (),
    .s_apb_PREADYPARITY  (),
    .s_apb_PSLVERRPARITY (),
    .parity_error_wdata  (),
    .parity_error_ctrl   ()
);
```

```
// Simple backend: return stored_data with no error and zeroed user
fields.
// Build the response from the parameters, not from literal widths --
the user
// fields are RUSER_WIDTH and BUSER_WIDTH wide, not a fixed 8 bits.
localparam int RUW = 4;   // RUSER_WIDTH
localparam int BUW = 4;   // BUSER_WIDTH

always_ff @(posedge apb_clk) begin
    if (!apb_rst_n) begin
        be_rsp_valid <= 1'b0;
    end else if (be_cmd_valid && be_cmd_ready) begin
        be_rsp_valid <= 1'b1;
        // {pslverr, prdata, pruser, pbuser}
        be_rsp_data  <= {1'b0, stored_data, {RUW{1'b0}}, {BUW{1'b0}}};
    end else if (be_rsp_ready) begin
        be_rsp_valid <= 1'b0;
    end
end
```

9.7 Design Notes

9.7.1 Skid Buffers

The stub uses skid buffers on both command and response paths: - **Command skid buffer:** Absorbs command while FSM transitions - **Response skid buffer:** Allows backend to push response before slave ready

9.7.2 Parity Implementation

When `ENABLE_PARITY=1`: - **Checking:** parity verified on incoming PWDATA (one bit per byte lane), PADDR (one bit for the whole address) and {PWRITE, PSTRB, PPROT} (one bit for the whole control group) - **Generation:** parity generated for outgoing PRDATA (one bit per byte lane), PREADY and PSLVERR - Each parity bit is the XOR reduction of the covered signals, an even-parity encoding - `parity_error_wdata` and `parity_error_ctrl` are combinational and qualified by `s_apb_PSEL && s_apb_PENABLE`; both are hard-tied to 0 when `ENABLE_PARITY=0`

9.7.3 Wake-up Handling

The `wakeup_request` input is registered before driving `s_apb_PWAKEUP`: - Provides clean synchronization - Single-cycle delay from request to PWAKEUP assertion

---

9.8 Related Documentation

- **APB5 Slave** - Full slave module
- **APB5 Master Stub** - Corresponding master stub

9.9 Navigation

- **← Back to APB5 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**